B.Tech. BCSE497J - Project-I

ATERNET: ADAPTIVE TEMPORAL EXPERT ROUTING NETWORK FOR PROBABILISTIC SOLAR POWER FORECASTING

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

<Reg. No.> **<STUDENT NAME 1>**

<Reg. No.> **<STUDENT NAME 2>**

<Reg. No.> **<STUDENT NAME 3>**

Under the Supervision of

<Project Guide Name>

<Designation>

School of Computer Science and Engineering (SCOPE)



October 2026

DECLARATION

I hereby declare that the project entitled **ATERNET: Adaptive Temporal Expert Routing Network for Probabilistic Solar Power Forecasting** submitted by me, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering* to VIT is a record of bonafide work carried out by me under the supervision of Prof. / Dr. <Project Guide Name>.

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place : Vellore

Date :

Signature of the Candidate

CERTIFICATE

This is to certify that the project entitled **ATERNET: Adaptive Temporal Expert Routing Network for Probabilistic Solar Power Forecasting** submitted by **<Student Name(s) (Reg. No.)>**, School of Computer Science and Engineering, VIT, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering*, is a record of bonafide work carried out by the candidate(s) under my supervision during Fall Semester 2026-2027, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place : Vellore

Date :

Signature of the Guide

Examiner(s)

<Head of the Department's Name>

Computer Science and Engineering

ACKNOWLEDGEMENTS

I am deeply grateful to the management of Vellore Institute of Technology (VIT) for providing me with the opportunity and resources to undertake this project. Their commitment to fostering a conducive learning environment has been instrumental in my academic journey. The support and infrastructure provided by VIT have enabled me to explore and develop my ideas to their fullest potential.

My sincere thanks to Dr. Jaisankar N, the Dean of the School of Computer Science and Engineering (SCOPE), for his unwavering support and encouragement. His leadership and vision have greatly inspired me to strive for excellence. The Dean's dedication to academic excellence and innovation has been a constant source of motivation for me. I appreciate his efforts in creating an environment that nurtures creativity and critical thinking.

I express my profound appreciation to <Head of the Department's Name>, the Head of the <Department Name>, for his/her insightful guidance and continuous support. His/her expertise and advice have been crucial in shaping the direction of my project. The Head of Department's commitment to fostering a collaborative and supportive atmosphere has greatly enhanced my learning experience. His/her constructive feedback and encouragement have been invaluable in overcoming challenges and achieving my project goals.

I am immensely thankful to my project guide, <Project Guide Name>, for his/her dedicated mentorship and invaluable feedback. His/her patience, knowledge, and encouragement have been pivotal in the successful completion of this project. My supervisor's willingness to share his/her expertise and provide thoughtful guidance has been instrumental in refining my ideas and methodologies. His/her support has not only contributed to the success of this project but has also enriched my overall academic experience.

Thank you all for your contributions and support.

<Name of the Candidate(s)>

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	Abstract	xiii
1.	**INTRODUCTION**	1
	1.1 Background	1
	1.2 Motivations	1
	1.3 Scope of the Project	2
2.	**PROJECT DESCRIPTION AND GOALS**	3
	2.1 Literature Review	3
	2.2 Research Gap	5
	2.3 Objectives	6
	2.4 Problem Statement	6
	2.5 Project Plan	6
3.	**TECHNICAL SPECIFICATION**	8
	3.1 Requirements	8
	3.1.1 Functional	8
	3.1.2 Non-Functional	8
	3.2 Feasibility Study	8
	3.2.1 Technical Feasibility	8
	3.2.2 Economic Feasibility	9
	3.2.3 Social Feasibility	9
	3.3 System Specification	9
	3.3.1 Hardware Specification	9
	3.3.2 Software Specification	9
4.	**DESIGN APPROACH AND DETAILS**	10
	4.1 System Architecture	10
	4.2 Design	12
	4.2.1 Data Flow Diagram	12
	4.2.2 Use Case Diagram	14
	4.2.3 Class Diagram	15
	4.2.4 Sequence Diagram	16

5. METHODOLOGY AND TESTING 19

5.1 Module Description 19

5.1.1 Data Acquisition and Preparation (Phase 1) 19

5.1.2 Temporal Backbone 21

5.1.3 Expert Pool 21

5.1.4 Adaptive Router 21

5.1.5 Probabilistic Decoder 22

5.1.6 Training Procedure 22

5.1.7 Interval Calibration 23

5.1.8 Baseline Models 24

5.1.9 Evaluation Metrics 24

5.1.10 Inference and Dashboard 24

5.1.11 Second Region and Simulated Site 24

5.2 Testing 25

5.2.1 Automated Tests 25

5.2.2 Experimental Protocol 25

5.2.3 Validity Checks 26

5.2.4 Defects Found and Corrected 26

6. PROJECT DEMONSTRATION 27

6.1 Overview Page 27

6.2 Is ATERNET Better? 29

6.3 India Demonstration (Simulated) 31

6.4 Uncertainty, Safety Checks and Further Pages 33

6.5 Running the Demonstration 37

7. RESULT AND DISCUSSION (COST ANALYSIS AS APPLICABLE) 38

7.1 Experimental Setup 38

7.2 Results on Yorkshire 38

7.3 Results on South West England 39

7.4 Development of the Model and Ablation Studies 40

7.5 Calibration and Coverage 42

7.6 Behaviour of the Forecasts and of the Uncertainty 44

7.7 Out-of-Sample Backtest 46

7.8 India Demonstration with Simulated Data 47

7.9 Additional Experiment: Denser Training Windows 48

7.10 Discussion 48

7.11 Cost Analysis 48

8. CONCLUSION 50

9. REFERENCES 51

APPENDIX A – SAMPLE CODE 55

List of Figures

Figure No.	Title	Page No.
2.1	Project plan and timeline (July to November 2026).	7
4.1	System architecture of the ATERNET forecasting system.	10
4.2	Architecture of the ATERNET model: backbone, expert pool, adaptive router and probabilistic decoder.	11
4.3	Data flow diagram, level 0 (context diagram).	13
4.4	Data flow diagram, level 1.	14
4.5	Use case diagram of the ATERNET forecasting system.	15
4.6	Class diagram of the main software components.	16
4.7	Sequence diagram of a live forecast request.	17
4.8	Sequence diagram of training, evaluation and calibration.	18
5.1	Flow of data acquisition and preparation.	20
5.2	One week (8–14 June 2026) of measured PV output in Yorkshire and the ensemble forecast of irradiance with the spread between the models.	21
5.3	Flow of training, evaluation and calibration.	23
6.1	Overview page of the dashboard (screenshot) (part 1 of 2)	28
6.2	Overview page of the dashboard (screenshot) (part 2 of 2)	29
6.3	Comparison page for the Yorkshire region (screenshot) (part 1 of 3)	30
6.4	Comparison page for the Yorkshire region (screenshot) (part 2 of 3)	31
6.5	Comparison page for the Yorkshire region (screenshot) (part 3 of 3)	31
6.6	India demonstration page with the simulated-data warning (screenshot) (part 1 of 2)	32
6.7	India demonstration page with the simulated-data warning (screenshot) (part 2 of 2)	33
6.8	Page showing how the model's unsureness relates to the real error (screenshot)	34
6.9	Built-in safety checks page (screenshot)	35

6.10	Forecast explorer page (screenshot)	36
6.11	Page showing how the four experts are trusted (screenshot)	37
7.1	Yorkshire: accuracy and quality of the forecast range for ATERNET and the baselines.	38
7.2	South West England: accuracy and quality of the forecast range.	40
7.3	nRMSE of each stage of the development (points = mean, whiskers = standard deviation; dashed line = LightGBM).	41
7.4	Change in nRMSE and Winkler score when a part is removed (grey = within noise; red = clearly worse; whiskers = ±2 standard deviations).	42
7.5	Share of daytime real values inside the P10–P90 range before and after calibration.	43
7.6	Reliability of the P10, P50 and P90 quantiles on daytime points of the Yorkshire test set.	43
7.7	Example forecasts for each weather situation (Yorkshire test set).	44
7.8	Average weight given to each expert (columns) in each real weather situation (rows) on the test set.	45
7.9	Average forecast error against the model's unsureness score and against the range width.	45
7.10	RMSE by forecast horizon for ATERNET and LightGBM (seed 42, Yorkshire).	46
7.11	Training and validation loss of the main model.	46
7.12	Backtest: P10–P90 range and best guess two hours ahead against the measured output.	47
7.13	Simulated Vellore site: forecast ranges and the naive 'same as now' guess against the simulated output.	47

List of Tables

Table No.	Title	Page No.
2.1	The twenty papers studied and how each relates to ATERNET.	4
2.2	Project objectives and their verification.	6
2.3	Phases of the project and their deliverables.	7
3.1	Functional requirements.	8
3.2	Non-functional requirements.	8
3.3	Hardware used for development and experiments.	9
3.4	Software environment.	9
4.1	Use cases and their actors.	15
5.1	Datasets used for the two measured regions.	19
5.2	Input and target tensors built in Phase 1.	19
5.3	The four experts.	21
5.4	Training configuration of the final model.	22
5.5	Baseline models used for comparison.	24
5.6	Evaluation metrics.	24
5.7	Automated test suite (74 tests, all passing).	25
5.8	Defects found during testing and how they were corrected.	26
6.1	Pages of the dashboard.	27
7.1	Yorkshire test results, mean ± standard deviation over training runs (lower nRMSE, nMAE, PINAW and Winkler are better). PICP counts night points; see Section 7.5.	38
7.2	Test nRMSE (%) of every training run on Yorkshire ('–' = not run).	39
7.3	South West England test results over three training runs.	39
7.4	Development of the model on Yorkshire (three seeds each).	40
7.5	Ablation studies on Yorkshire (three seeds each). Δ is the change relative to the reference; 'worse without it' means removing the part made the result worse beyond about two standard deviations.	41
7.6	Other training options tested (Yorkshire, three seeds each).	42

7.7	Effect of conformal calibration on one trained model (seed 42) per site; target coverage 0.80.	43
7.8	Results by weather situation (Yorkshire, seed 42, test set).	44
7.9	Backtest of the live pipeline, Yorkshire, 1–16 October 2026.	46
7.10	Simulated Vellore plant (1 MWp), one training run each; test windows: 1,805.	47
7.11	Effect of doubling the number of training windows (Yorkshire).	48
7.12	Cost and resource summary.	49

List of Abbreviations

ACE	Average Coverage Error
API	Application Programming Interface
ATERNET	Adaptive Temporal Expert Routing Network
CQR	Conformalized Quantile Regression
CRPS	Continuous Ranked Probability Score
DFD	Data Flow Diagram
EMA	Exponential Moving Average
ERA5	ECMWF Reanalysis, fifth generation
GBDT	Gradient-Boosted Decision Trees
GEM	Global Environmental Multiscale model
GFS	Global Forecast System
GHI	Global Horizontal Irradiance
ICON	Icosahedral Nonhydrostatic model
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
MLP	Multi-Layer Perceptron
nMAE	Normalised Mean Absolute Error
nRMSE	Normalised Root Mean Square Error
NWP	Numerical Weather Prediction
PICP	Prediction Interval Coverage Probability
PINAW	Prediction Interval Normalised Average Width
PV	Photovoltaic
RMSE	Root Mean Square Error
R^2	Coefficient of determination
UKMO	United Kingdom Meteorological Office model

Symbols and Notations

α	Mixing weight between the two uncertainty signals (0 to 1)
β	Learned sensitivity of the gate temperature to the uncertainty
γ	Learned sensitivity of the interval width to the uncertainty
δ	Learned scale of the residual gate (initialised to 0)
σ_NWP	Spread between the weather-forecast models
σ_expert	Disagreement between the four experts
σ_total	Combined uncertainty used by the gate and the decoder
T	Gate temperature
τ	Quantile level of the pinball loss (0.1, 0.5, 0.9)
$\hat{y}$	Forecast of an expert or of the router
h_expert, h_sigma, h_nwp	Embeddings produced by the backbone
P10, P50, P90	10th, 50th and 90th percentile forecasts
X_seq, X_nwp, X_sigma	Input tensors (history, forecast weather, forecast-spread features)
θ_ema	Exponentially averaged model weights

ABSTRACT

Solar photovoltaic (PV) generation depends on weather that cannot be predicted exactly, so grid operators need forecasts that state not only how much power to expect but also how sure the forecast is. This project presents ATERNET (Adaptive Temporal Expert Routing Network), a deep-learning model that forecasts regional solar generation for the next eight hours at 30-minute resolution and returns three quantiles (P10, P50 and P90) that together form an 80% prediction interval. The model combines a PatchTST-style Transformer backbone, a pool of four weather-regime experts, an uncertainty-aware router that fuses the disagreement between numerical weather prediction (NWP) models with the disagreement between the experts, and a probabilistic decoder whose interval width grows with that uncertainty.

ATERNET was trained and tested on measured regional PV output from Sheffield Solar PV_Live for two English regions (Yorkshire and South West England, January 2023 to September 2026) together with forecasts from five NWP models obtained through Open-Meteo. All comparisons use several random training runs, and a difference is reported only when it exceeds the run-to-run variation. Our first design, with a learned routing gate, was less accurate and far less stable than a plain average of the experts (nRMSE 4.39 ± 0.20% against 4.12%). A residual gate that starts as uniform routing, a zero-initialised linear shortcut from the latest observation and exponential averaging of the weights fixed this and brought the error to 3.70 ± 0.02%, level with LightGBM (3.72 ± 0.01%) and clearly better than PatchTST (4.16 ± 0.00%); its forecast ranges obtained a better Winkler score than both. On South West England, with no re-tuning, ATERNET was ahead of LightGBM (2.63 ± 0.01% against 2.69 ± 0.00%) and PatchTST (2.92 ± 0.03%).

Rigorous checking also corrected our own early results. Coverage statistics that include night-time hours overstated how well the ranges held; in daylight the raw ranges contained the real value only 67% of the time instead of 80%. After fixing the conformal calibration step and applying it to daytime points, daytime coverage reached 79% and 80% on the two regions. Ablation studies showed that only the dense expert loss measurably improved the results; the adaptive gate, the temperature control and regime specialisation did not, and we report this openly.

Because no open measured PV record exists for the VIT Vellore area, we also built a clearly labelled simulated demonstration for Vellore, driven by real weather data. The work is delivered with an automated test suite of 74 tests and an interactive dashboard that explains every result in plain language.

CHAPTER 1
INTRODUCTION

1.1 Background

Solar photovoltaic (PV) power is among the fastest-growing sources of electricity worldwide. Unlike a conventional power station, a PV plant cannot be dispatched on demand: its output follows the sun and is cut sharply by clouds. When the share of solar power in a grid is small this variability is absorbed without difficulty, but as the share grows, system operators must anticipate how much solar energy will be available in the next minutes, hours and days in order to schedule other generation, keep reserves ready, trade energy and maintain the balance between supply and demand [1, 2].

Forecasts of PV power are produced from three kinds of information: recent measurements of the plant's own output, satellite and sky observations, and numerical weather prediction (NWP), in which physics-based atmospheric models are run on supercomputers to predict irradiance, cloud cover and temperature [2]. For horizons of a few hours, the most useful forecasts combine the latest measurements with NWP output, because the measurements describe what is happening now while the NWP describes what the atmosphere is about to do.

A single number is not enough for decisions. If a forecast says 300 MW but the true value could easily lie anywhere between 150 MW and 450 MW, an operator will plan very differently than if the plausible range is 280 to 320 MW. For this reason the research community has moved from point forecasts to probabilistic forecasts, which describe the whole distribution of possible outcomes [1, 3]. A common and practical form is a prediction interval, for example the range between the 10th and 90th percentiles (P10 and P90) together with a median (P50). Such an interval is useful only if it is reliable (the real value falls inside it about as often as promised) and sharp (it is no wider than necessary).

1.2 Motivations

Modern deep-learning models, and in particular Transformer networks, have improved forecasting accuracy in many domains, but three practical problems remain for solar forecasting. First, the accuracy of a model depends strongly on the weather situation: a clear sunny midday, a ramp at sunrise, a uniformly cloudy day and a day of broken clouds behave very differently, and a single network tends to be averaged over them. Second, the uncertainty of a forecast has two different sources, namely disagreement between weather models and the model's own uncertainty, and most systems either ignore one of them or learn a single opaque confidence value. Third, results in this field are often reported from a single training run and with statistics that look better than they are, which makes it hard to know which improvements are real.

These observations motivated us to design a forecaster that (i) separates the weather situations into specialised experts, (ii) combines the two sources of uncertainty explicitly and uses them to decide both how much to trust each expert and how wide to draw the forecast interval, and (iii) is evaluated honestly, with several random starts, strong baselines and an explicit separation of what the data do and do not support. We also wanted the work to be

demonstrable for our own campus at VIT Vellore.

1.3 Scope of the Project

The project covers the complete pipeline for regional PV forecasting: data acquisition, cleaning and feature engineering, model design and training, comparison with three baseline models, calibration of the forecast intervals, an interactive dashboard and an automated test suite. Specifically:

- **Forecast task.** Predict the regional PV output for the next 16 half-hours (8 hours) as P10, P50 and P90, given the previous 24 hours of measured output and the weather forecasts for the past and the next 8 hours.
- **Data.** Measured half-hourly regional generation from Sheffield Solar PV_Live for the Yorkshire region (NPG Yorkshire) and for South West England (WPD South West), and five-model weather forecasts from Open-Meteo, from 1 January 2023 to 30 September 2026.
- **Evaluation.** Accuracy (nRMSE, nMAE, R^2), interval quality (coverage, width, Winkler score), comparison with LightGBM, PatchTST and LSTM, ablation studies, multiple random seeds and an out-of-sample backtest on October 2026.
- **Demonstration for India.** A clearly labelled simulated plant at VIT Vellore, because no open measured PV record exists for the area.

The project does not cover forecasting for individual panels, horizons beyond eight hours, grid control or market bidding, and it makes no claim about real measured performance in India, for which we were unable to obtain data (Section 7.8).

CHAPTER 2
PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

We studied twenty recent papers on probabilistic forecasting of solar power and of other energy series (Table 2.1). They are grouped below by the idea they contribute, and each group is related to the design of ATERNET. Some of the papers concern other energy series (wind, load, curtailment, thermal energy); this is stated where it matters, because their methods can transfer but their results do not.

Quantile-based probabilistic forecasting of PV. Most probabilistic PV models output quantiles trained with the pinball loss [4, 5]. Lo et al. [1] propose a quantile-regression procedure that skips training the separate upper- and lower-bound models when a site's output is stable. He et al. [6] decompose the power series with adaptive variational mode decomposition, predict with a bidirectional LSTM with feature attention, and add an asymmetric monotonicity penalty so that quantiles do not cross; Huang et al. [7] likewise use a non-crossing joint quantile loss for a distributed PV cluster. Zhao et al. [2] feed multi-source NWP forecasts to a two-stage ensemble in which diverse base learners produce quantiles and an attention-based meta-learner combines them under a non-crossing quantile generation strategy. Phan et al. [8] combine a Transformer with lower-upper-bound estimation, XGBoost-based imputation of missing data and a GRU that corrects residual bias, and He et al. [9] use a generative adversarial network to learn the joint distribution of measurements and NWP, with a multi-branch CNN-LSTM and quantile regression with kernel density estimation. In ATERNET the quantiles cannot cross by construction, because the interval is built from positive half-widths around the median, so no penalty term is needed.

Dependence on the weather situation. Forecast difficulty depends on the weather. Lyu and Eftekharnejad [3] combine copula theory with an XGBoost classifier so that the dependence between meteorological variables changes with the weather type, and report gains of up to 60% under non-sunny conditions. Wang et al. [10] use a Transformer point forecast, an autoencoder that recognises fluctuation patterns and a Clayton copula for the joint distribution of forecast values and errors. Related ideas appear in other energy domains: TADNet [11] treats a series as a regular component plus a residual whose fluctuation interval is quantified, and Zhao et al. [12] add Gaussian-process residual learning to a Transformer for multi-energy load forecasting. These works support treating different situations differently, which ATERNET does with its regime experts.

Mixtures of experts and gating. A mixture of experts splits a task between specialised sub-models and a gating network that combines them [13, 14]. Wang et al. [15] use a context-gated mixture of experts to weight LSTM, GRU, Transformer and tree-based learners for PV forecasting, with a gate driven by input features, local prediction variance and historical residuals, followed by a residual corrector and a conformal interval. Heidrich et al. [16] place a mixture of experts with SwiGLU feed-forward experts and an auxiliary load-balancing loss inside a patch-based Transformer for curtailment forecasting, and Seman et al. [17] use sparse expert Transformers with measures against expert collapse for thermal energy forecasting. Load-balancing penalties are used to keep all experts in use [14, 16]. These studies show that

gating is attractive, but they do not settle whether a learned gate beats a plain average of the experts for regional PV, which the experiments in Chapter 7 test directly.

Calibrated intervals. Conformal prediction gives distribution-free intervals. Romano et al. [18] conformalise quantile regression, and Jensen et al. [19] extend this to time series with a bootstrap ensemble of quantile regressors and asymmetric conformity scores, obtaining sharper valid intervals. Conformal steps are also part of the PV and curtailment frameworks of [15] and [16]. We use the quantile-regression conformity score of Romano et al. per weather regime (Section 5.1.7).

Spatial and image-based methods. Another group of recent papers uses spatial structure or imagery. Graph networks model the dependence between PV sites: Wang et al. [20] with a dynamic graph and a loss that aligns both the shape and the amplitude of the forecast, Lin et al. [21] with a convolutional graph network and parameter transfer on 127 plants, Huang et al. [7] with spatial-temporal attention, and Song et al. [22] with random coarse-graph attention that also handles missing data across 1,649 sites. Image-based work predicts cloud motion: Rastgoo et al. [23] use a diffusion model on sky-image sequences with an evidence-lower-bound loss, and Wang et al. [24] a diffusion model on satellite images with a neural stochastic differential equation, whose image ensembles are combined with PV history and NWP. These methods need plant-level networks or image streams; ATERNET needs only the regional PV series and NWP forecasts. For wind power, Abedinia et al. [25] estimate the parameters of a spline quantile function with a CNN-LSTM trained with an adaptive weighted CRPS, an alternative to predicting fixed quantiles.

Sequence models and boosted trees. Recurrent networks, in particular the long short-term memory network (LSTM) [26], were for years the standard deep model for time series. The Transformer [27] replaced recurrence by attention, and PatchTST [28] adapted it to forecasting by splitting each series into short overlapping patches that are treated as tokens. We use a PatchTST-style encoder as the backbone of ATERNET and a vanilla PatchTST predicting quantiles as a baseline. LightGBM [29] is a fast gradient-boosting library that remains very competitive on tabular data; it does not model time explicitly, so features describing the recent past and the forecast weather must be supplied. Because of its strength we treat it as the principal baseline.

Scoring and training stability. An interval is assessed by its coverage (PICP), its width (PINAW) and by the Winkler score [30], which adds a penalty proportional to the amount by which the observation lies outside the interval, so that a model cannot look good simply by drawing very wide intervals. ReZero [31] multiplies each residual branch by a learnable scalar initialised to zero so that a network starts as the identity and only departs from it when that helps. Averaging the weights along the training trajectory [32], today usually implemented as an exponential moving average (EMA), is a cheap way to obtain a smoother and often more accurate model. Both ideas proved decisive in our work (Section 7.4).

Table 2.1: The twenty papers studied and how each relates to ATERNET.

Work	Approach (as stated in the paper)	Relation to ATERNET
Wang et al. 2026 [20]	Dynamic graph network over distributed PV sites; shape-amplitude loss (dynamic time warping + MSE); quantile regression	Needs several sites; ATERNET models one regional series

Work	Approach (as stated in the paper)	Relation to ATERNET
Lo et al. 2025 [1]	Quantile regression that skips training upper and lower bounds when output is stable; missing-data imputation	ATERNET always outputs three quantiles
He et al. 2026 [6]	Adaptive VMD, BiLSTM with feature attention, asymmetric monotonicity penalty against quantile crossing	ATERNET prevents crossing by construction
Wang et al. 2025 [15]	Context-gated mixture of experts over LSTM, GRU, Transformer and tree learners; residual corrector; conformal interval; no NWP	Closest gating idea; ATERNET gates on NWP spread and expert disagreement
Zhao et al. 2026 [2]	Two-stage ensemble with meta-learner on multi-source NWP; skip connections; non-crossing quantile generation	Closest NWP use; ATERNET uses the spread between NWP models as uncertainty
He et al. 2026 [9]	Similar-moment selection; GAN on measurements + NWP; multi-branch CNN-LSTM; quantile regression with KDE	Data augmentation; ATERNET uses no generative model
Heidrich et al. 2026 [16]	Patch-based Transformer with SwiGLU mixture of experts, load-balancing loss and conformal inference (curtailment)	Same building blocks, different series
Lyu and Eftekharnejad 2024 [3]	Copula theory with an XGBoost weather classifier for changing dependencies	Weather-dependent modelling, like our regimes
Lin et al. 2025 [21]	Dynamic convolutional graph network; parameter transfer; regional PV from 127 plants	Needs plant-level data
Huang et al. 2026 [7]	Spatial-temporal attention and graph convolution; non-crossing joint quantile loss	Cluster graph; ATERNET uses no graph
Rastgoo et al. 2025 [23]	Diffusion model on sky-image sequences; ELBO loss; ultra-short term	Needs sky images
Seman et al. 2026 [17]	Hierarchical VMD, architecture search, sparse expert Transformers (thermal energy)	Different series; shares expert idea
Phan et al. 2024 [8]	Transformer-LUBE; XGBoost imputation; GRU residual correction; day-ahead PV with NWP	Point-and-bounds design; ATERNET is quantile-based
Jensen et al. 2024 [19]	Ensemble conformalized quantile regression with asymmetric conformity scores	Related calibration; we use the CQR score per regime
Song et al. 2025 [22]	Random coarse-graph attention and imputation across 1,649 PV sites	Large-scale graph; ATERNET is single-region
Wang et al. 2026 [24]	Diffusion-based satellite image prediction; neural SDE; multimodal ensemble with PV and NWP	Needs satellite images
Zhao et al. 2024 [12]	Hybrid-attention Transformer with Gaussian-process residual learning (multi-energy load)	Residual modelling idea; different series
Ye et al. 2024 [11]	Regular component plus residual fluctuation interval; convolution with attention and gating	Decomposition of the series; different series
Wang et al. 2026 [10]	Transformer, autoencoder fluctuation patterns and Clayton copula for forecast intervals	Pattern-dependent intervals; copula-based
Abedinia et al. 2024 [25]	CNN-LSTM estimating a spline quantile function; adaptive weighted CRPS (wind power)	Alternative to fixed quantiles; wind

2.2 Research Gap

From these papers and from our preliminary experiments we identified the following gaps, which define the contribution of this project:

1. **Uncertainty from disagreement.** Among the twenty papers, we could not find one that uses the disagreement between several NWP models, or between several experts, as an explicit uncertainty signal that also sets the width of the interval and the sharpness of a gate. Multi-source NWP is used as input in [2] and gating on prediction variance in [15], but not the two together.
2. **Value of gating.** Mixtures of experts and gating are reported for PV, curtailment and

thermal energy [15, 16, 17], but whether a learned gate **actually beats a plain average** of the experts, and whether it is stable across training runs, needs a direct test for regional PV.

3. **Calibration where it matters.** PV series contain exact zeros at night, where a zero-width interval is trivially correct. Coverage should therefore be assessed, and conformal calibration fitted, on **daytime** points (Section 5.1.7); we found that all-hours coverage hides under-coverage in daylight.
4. **Data requirements.** Graph and image methods [7, 20, 21, 22, 23, 24] need plant-level networks or image streams. A model that works from a regional aggregate and NWP forecasts is useful where these are not available.
5. **Reliable comparison.** Differences of a few percent between models can be smaller than the run-to-run variation of training. We use several random starts, strong baselines and a second region, and call a difference real only when it exceeds that variation.
6. **Regions without open data.** For areas such as the one around our campus, a clearly labelled **simulated demonstration** is needed that does not pretend to be a real validation.

2.3 Objectives

The objectives of the project, with the way each one is verified, are:

Table 2.2: Project objectives and their verification.

No.	Objective	How it is verified
O1	Route between weather-regime experts using the combined uncertainty (context-gated routing).	Gate behaviour (Fig. 7.8), ablations without the gate (Table 7.5)
O2	Make the width of the forecast interval depend on the uncertainty (uncertainty-conditioned decoding).	Correlation of width and error (Fig. 7.9), width-ablation, safety check 2
O3	Train all parts jointly, keeping every expert learning (dense expert loss).	Ablation without the dense expert loss (Table 7.5)
O4	Evaluate per weather regime and per forecast horizon.	Tables 7.8 and Fig. 7.10
O5	Compare honestly with strong baselines over several random seeds and on a second region.	Tables 7.1 and 7.3, Figures 7.1 and 7.2
O6	Deliver a tested, reproducible system and an understandable dashboard, including a simulated demonstration for VIT Vellore.	74 automated tests (Table 5.7), Chapter 6, Section 7.8

2.4 Problem Statement

Given the measured generation of a PV region over the last 24 hours and multi-model weather forecasts for the last 24 hours and the next 8 hours, produce for each of the next 16 half-hours a median forecast and an 80% prediction interval (P10 to P90) such that (a) the median is as accurate as the best available baseline, (b) the real value falls inside the interval about 80% of the time in daylight, (c) the interval is as narrow as possible, (d) the result is stable across random training starts, and (e) the system can be explained to a non-specialist.

2.5 Project Plan

The work was carried out in the phases shown in the plan below. The first model, with a data pipeline and a synthetic data source, was built in August 2026; real data were integrated from

September; the main experiments, improvements and corrections took place in October 2026, followed by the dashboard, the second region, the simulated India demonstration and this report.

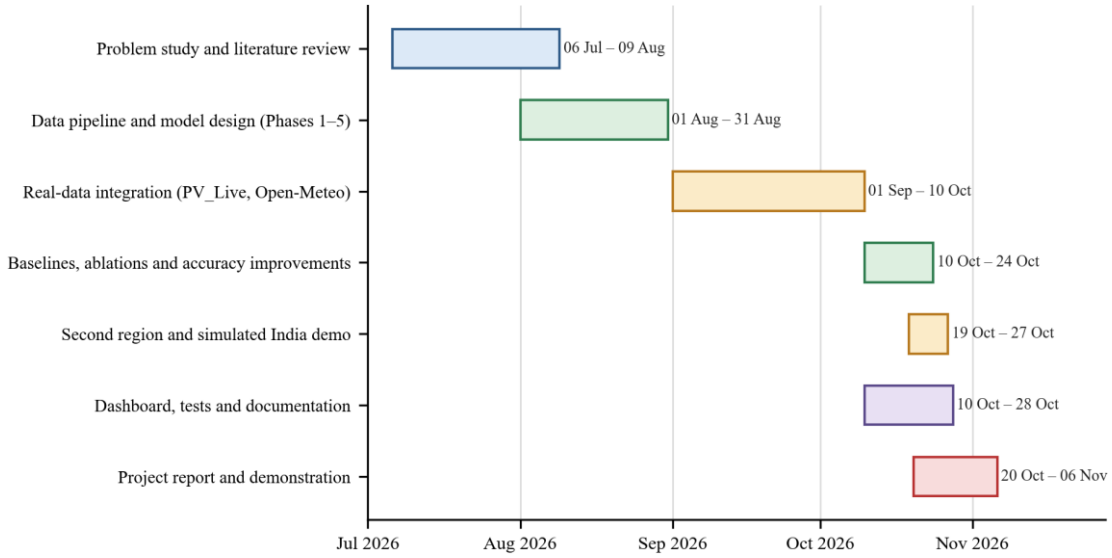


Figure 2.1: Project plan and timeline (July to November 2026).

Table 2.3: Phases of the project and their deliverables.

Phase	Main deliverable
Problem study, literature review	Problem definition, design of the five-phase architecture
Pipeline and model (Phases 1–5)	Data pipeline, backbone, experts, router, decoder, first training run
Real-data integration	PV_Live and Open-Meteo sources, corrected units and capacity, longer weather input
Baselines, ablations, improvements	LightGBM/PatchTST/LSTM, multi-seed protocol, residual gate, shortcut, EMA
Second region and India demo	South West England experiment, simulated Vellore site
Dashboard, tests, documentation	Streamlit dashboard, 74 tests, this report

CHAPTER 3
TECHNICAL SPECIFICATION

3.1 Requirements

3.1.1 Functional

Table 3.1: Functional requirements.

ID	Requirement
F1	Download measured regional PV generation and installed capacity (PV_Live) for a chosen region and period.
F2	Download multi-model weather forecasts (Open-Meteo) and align them to the PV time grid.
F3	Clean the data, derive features (clear-sky index, time encodings, forecast-spread features, regime labels) and build 24 h to 8 h windows without leaking future information.
F4	Train the ATERNET model jointly with early stopping, checkpointing and optional weight averaging.
F5	Output P10, P50 and P90 for 16 steps with the guarantee $P10 \leq P50 \leq P90$.
F6	Evaluate with nRMSE, nMAE, R^2, PICP, PINAW, Winkler score and skill against persistence, per regime and per horizon.
F7	Train and evaluate the LightGBM, PatchTST and LSTM baselines under the same protocol.
F8	Calibrate the intervals per weather regime on daytime validation points.
F9	Run experiments over several random seeds and aggregate means and standard deviations.
F10	Provide live forecast and backtest commands, and an interactive dashboard of the results.
F11	Support a second region and a clearly flagged simulated site through configuration only.

3.1.2 Non-Functional

Table 3.2: Non-functional requirements.

ID	Requirement	How it is met
N1	Reproducibility	Fixed seeds, saved configuration and checkpoints, results written to files, one command per experiment
N2	Scientific validity	Chronological split, scalers fitted on training data only, multi-seed reporting, results read from files
N3	Robustness	Retry and automatic request-splitting for downloads; gap detection; automatic early stopping
N4	Isolation	Environment variables move the data, outputs and checkpoints to a separate workspace so experiments cannot overwrite each other
N5	Understandability	Dashboard in plain language with a glossary and colour-coded comparison
N6	Efficiency	A full training run takes about 10 minutes on a laptop CPU
N7	Maintainability	Modular code (about 7,800 lines of Python in src, config and tests) and 74 automated tests
N8	Honesty of reporting	Wins are claimed only beyond ~2 seed standard deviations; simulated data are labelled everywhere

3.2 Feasibility Study

3.2.1 Technical Feasibility

All required components are available as mature open-source software (Python, PyTorch, pandas, LightGBM, Streamlit) and the data sources are public web services. The model has about 968,000 trainable parameters and the training set has about 11,500 windows, so a full training run fits comfortably on an ordinary laptop without a GPU. This was confirmed in

practice: all experiments in this report, including more than 60 training runs, were executed on a laptop with an Intel Core i5-12450H processor and 7.7 GB of memory. The main technical risks were the quality of the open data (handled by validation and tests) and the memory limit of the laptop (handled by running one job at a time).

3.2.2 Economic Feasibility

The project uses no paid software, data or cloud services. Sheffield Solar PV_Live and Open-Meteo are free for research use, and every library is open source. The only cost is the author's time and the electricity of a laptop running for a few hours per experiment (Section 7.11). A deployment for an operator would add the cost of a data feed and hosting, but the model is small enough to run on a single inexpensive server or even on the operator's existing computers.

3.2.3 Social Feasibility

Better solar forecasts help grids integrate more renewable energy, reduce the need for fossil-fuel reserve generation and lower energy costs. Interval forecasts that are honest about uncertainty reduce the risk of over-reliance on a single number. The system works on aggregated regional data and uses no personal information. The main social risk is misuse of the simulated Indian demonstration as though it were measured evidence; for that reason it is labelled as simulated in the code, in the dashboard and in this report.

3.3 System Specification

3.3.1 Hardware Specification

Table 3.3: Hardware used for development and experiments.

Item	Specification used
Processor	Intel Core i5-12450H (8 cores, 12 threads)
Memory	7.7 GB RAM
Graphics	NVIDIA GeForce GTX 1650 present, but PyTorch was the CPU build; all training ran on the CPU
Storage	A few GB for cached downloads, datasets, checkpoints and results
Network	Internet access for data downloads and live forecasts

3.3.2 Software Specification

Table 3.4: Software environment.

Component	Version / description
Operating system	Windows 11 (10.0.26300)
Language	Python 3.11.9
Deep learning	PyTorch 2.5.1 (CPU) [33]
Gradient boosting	LightGBM 4.7.0 [29]
Data handling	NumPy 2.2.6, pandas 2.2.3, PyArrow 22.0.0, SciPy 1.15.1, scikit-learn 1.8.0
Plotting and dashboard	Matplotlib 3.9.2, Plotly 6.1.0, Streamlit 1.50.0 [34]
Testing	pytest 9.1.1 (74 tests)
Data services	Sheffield Solar PV_Live API [35]; Open-Meteo forecast archive and ERA5 reanalysis API [36, 37]
Version control	Git (29 commits at the time of writing)

CHAPTER 4
DESIGN APPROACH AND DETAILS

4.1 System Architecture

The system is organised in four layers (Figure 4.1). The **data layer** downloads measured PV output and forecast weather, cleans and aligns them, derives features and cuts them into windows. The **model and experiment layer** contains the ATERNET network, its trainer, the baseline models, the evaluation code and the calibration step. The **application layer** exposes the results through a command-line forecaster and an interactive dashboard, and a small results store makes the numbers available to the dashboard without re-running experiments.

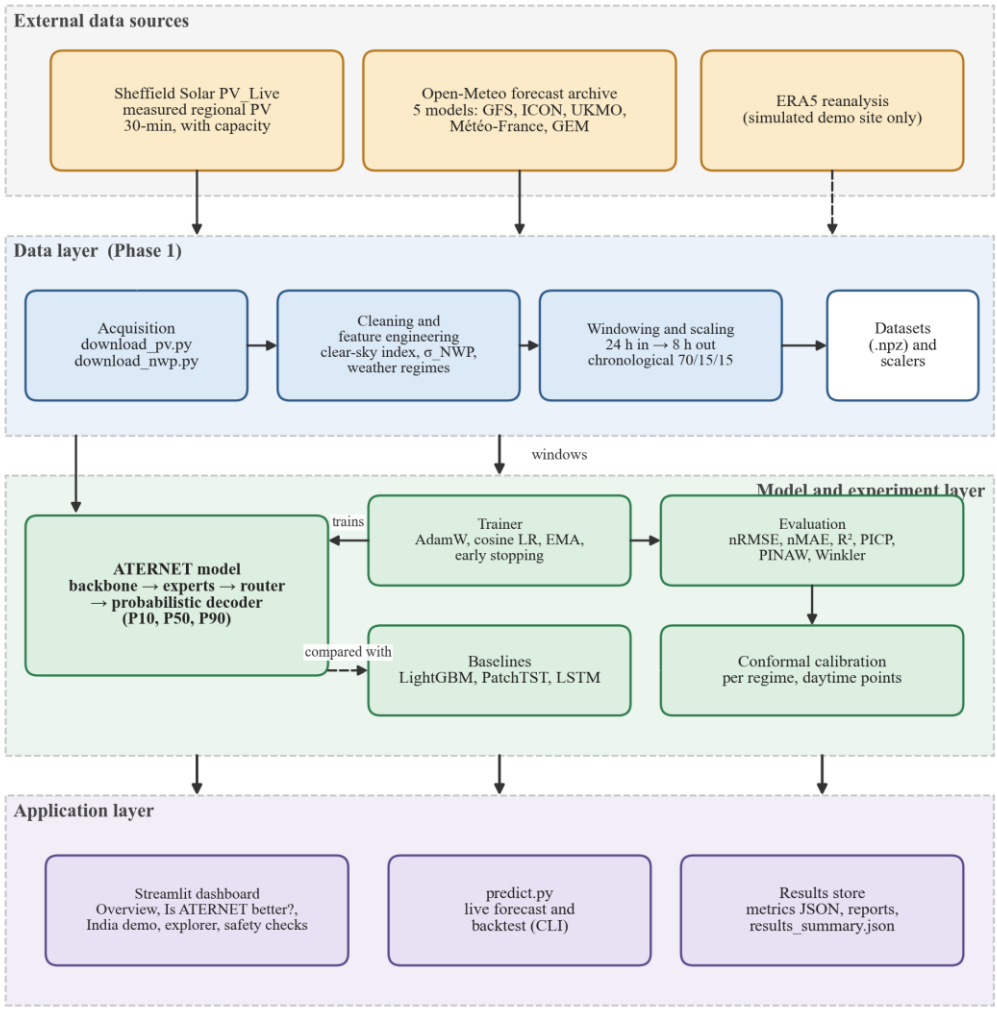


Figure 4.1: System architecture of the ATERNET forecasting system.

The ATERNET network (Figure 4.2) is built from five phases that are trained together. Three input tensors enter: the last 24 hours of measured and derived signals (X_seq, 48 steps × 10 features), the weather forecasts for the past 24 hours and the next 8 hours (X_nwp, 64 × 4) and the forecast-spread features that describe how much the NWP models disagree (X_sigma, 48 × 6).

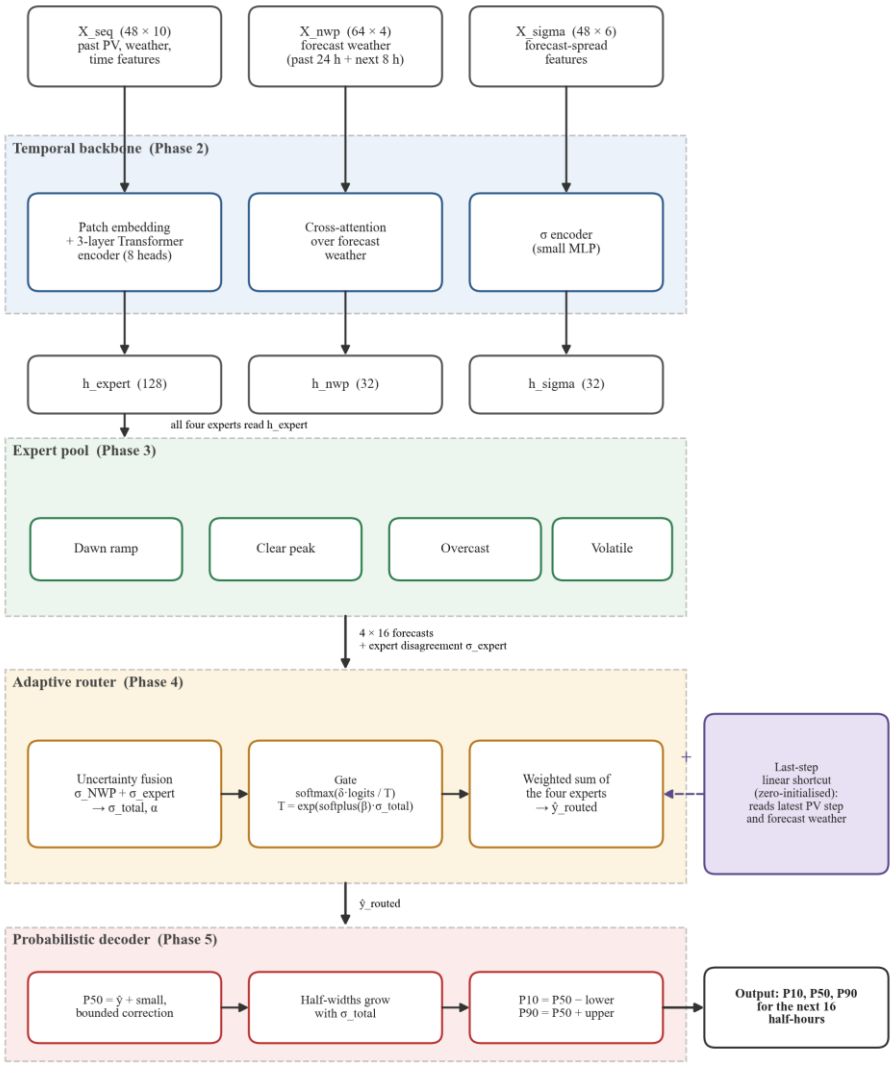


Figure 4.2: Architecture of the ATERNET model: backbone, expert pool, adaptive router and probabilistic decoder.

- **Phase 2, temporal backbone.** The measured sequence is cut into overlapping patches (8 steps, stride 4), embedded, given a learnable position and passed through a three-layer Transformer encoder with eight attention heads. A cross-attention block lets the PV patches look at the forecast-weather patches, and a small network encodes the forecast-spread features. The backbone produces three embeddings: h_expert (128 values) for the experts, h_nwp (32) for the decoder and h_sigma (32) for the router.
- **Phase 3, expert pool.** Four expert networks, one for each weather regime (morning ramp-up, clear midday, overcast, patchy clouds), each predict the 16 future values from h_expert. The experts differ in capacity, loss and physical constraint (Table 5.3). Their disagreement is measured as σ_expert, the standard deviation across experts, computed on detached predictions so that it cannot be gamed by the experts.
- **Phase 4, adaptive router.** Two uncertainty signals are combined: σ_NWP, the spread between the NWP models, and σ_expert, the disagreement between experts. A learned mixing weight α depends on the embedding, so the model decides when to trust which signal. The combined σ_total sets the temperature of a softmax gate that weights the

experts, and also controls the width of the final interval.

- **Phase 5, probabilistic decoder.** A bounded correction is added to the routed forecast to give the median (P50). Separate lower and upper half-widths that increase with σ_total give P10 and P90, so the quantiles can never cross.

The uncertainty mixing and the temperature are given by

$$\sigma_total = \alpha \cdot \tilde{\sigma}_NWP + (1 - \alpha) \cdot \tilde{\sigma}_exp\ , \quad \alpha = sigmoid(w^T h_expert + b) \tag{4.1}$$

$$T = exp(\ softplus(\beta) \cdot \sigma_total\ )\ , \quad weights = softmax(\ \delta \cdot z\ /\ T\ ) \tag{4.2}$$

where σ̃ denotes a normalised uncertainty, z the gate's raw scores for the four experts, β a learned sensitivity and δ a learned scale (explained below). A large σ_total therefore raises the temperature T and spreads the votes across experts, as a hedge. The routed forecast and the interval are

$$\hat{y}_routed = \Sigma_k\ weights_k \cdot \hat{y}_k + S(x_last,\ x_nwp,future) \tag{4.3}$$

$$lower_h = softplus(a_h + softplus(\gamma_h) \cdot \sigma_total) \quad (upper_h\ analogous) \tag{4.4}$$

$$P10 = clip(P50 - lower)\ , \quad P90 = clip(P50 + upper) \tag{4.5}$$

The term S in the first equation is the **last-step shortcut**, a single linear layer from the latest observed step and the forecast weather of the next eight hours to the 16 outputs. Its weights start at zero, so at the start of training the model is identical to one without it.

Three structural guarantees are built into the design and checked by automated tests: (G1) P10 ≤ P50 ≤ P90 always holds, because the widths are positive by construction; (G2) the interval width is non-decreasing in σ_total; and (G3) the gate softens (spreads its votes) when σ_total grows.

The two design changes that came from our experiments are explained in Chapter 7. First, the original gate, which learned its scores directly, was replaced by a **residual gate** in which the scores are multiplied by a learnable scalar δ initialised to zero, so that routing starts as an exact uniform average and departs from it only if that lowers the validation loss (the same principle as ReZero [31]). Second, the optional shortcut and an exponential moving average of the weights (Section 5.1.6) were added.

4.2 Design

4.2.1 Data Flow Diagram

The context diagram (Figure 4.3) shows the system as a single process with its external entities: the two data services supply measured PV and weather; researchers supply configuration and training requests; reviewers and grid analysts receive results, comparisons and forecasts with ranges.

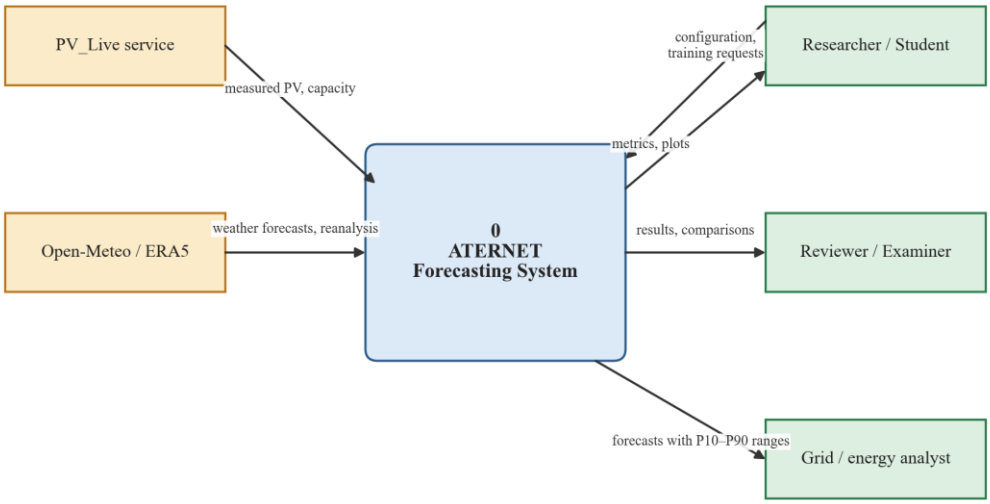


Figure 4.3: Data flow diagram, level 0 (context diagram).

The level-1 diagram (Figure 4.4) decomposes the system into six processes and four data stores. Process 1.0 acquires raw data into D1; process 2.0 cleans, engineers features and builds windows into D2; process 3.0 trains the model and saves the best weights into D3; process 4.0 evaluates and calibrates and writes metrics to D4; process 5.0 produces live or backtest forecasts from the stored model; process 6.0 presents everything in the dashboard.

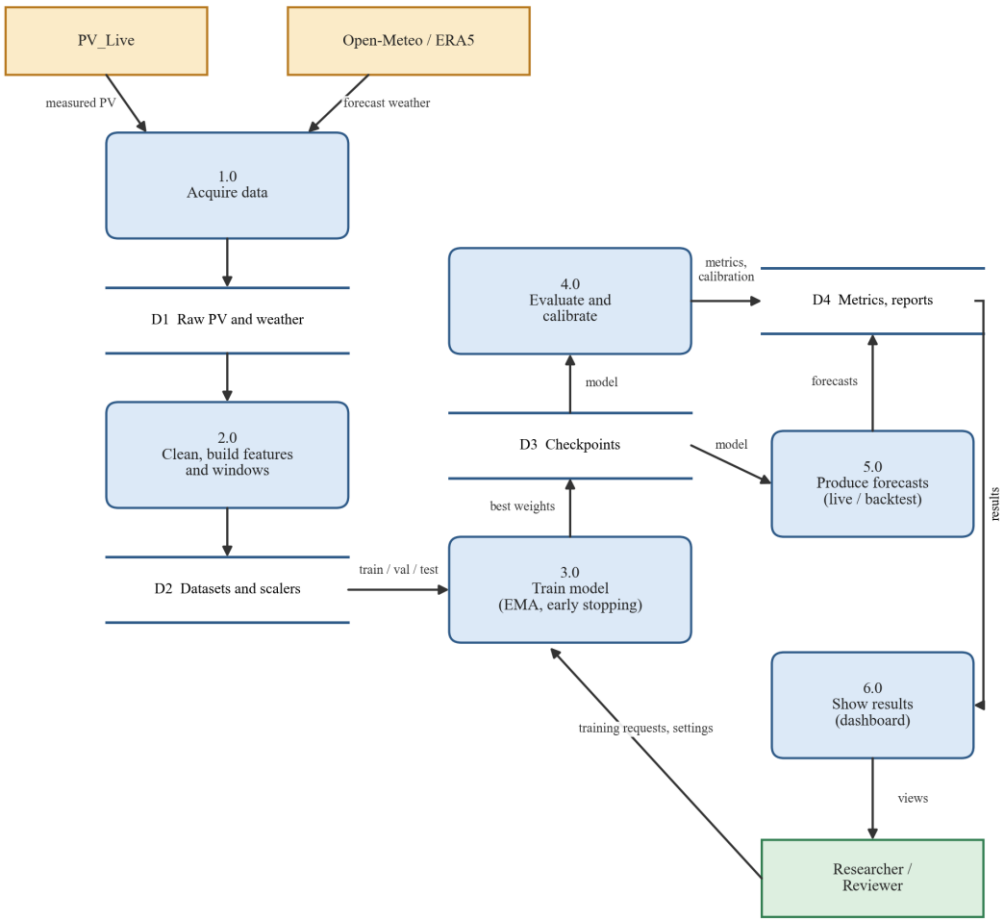


Figure 4.4: Data flow diagram, level 1.

4.2.2 Use Case Diagram

Figure 4.5 shows the actors and use cases. The researcher or student prepares data, trains and evaluates models and runs studies; the reviewer or examiner explores results; the grid or energy analyst consumes forecasts. The two data services take part in the use cases that need external data.

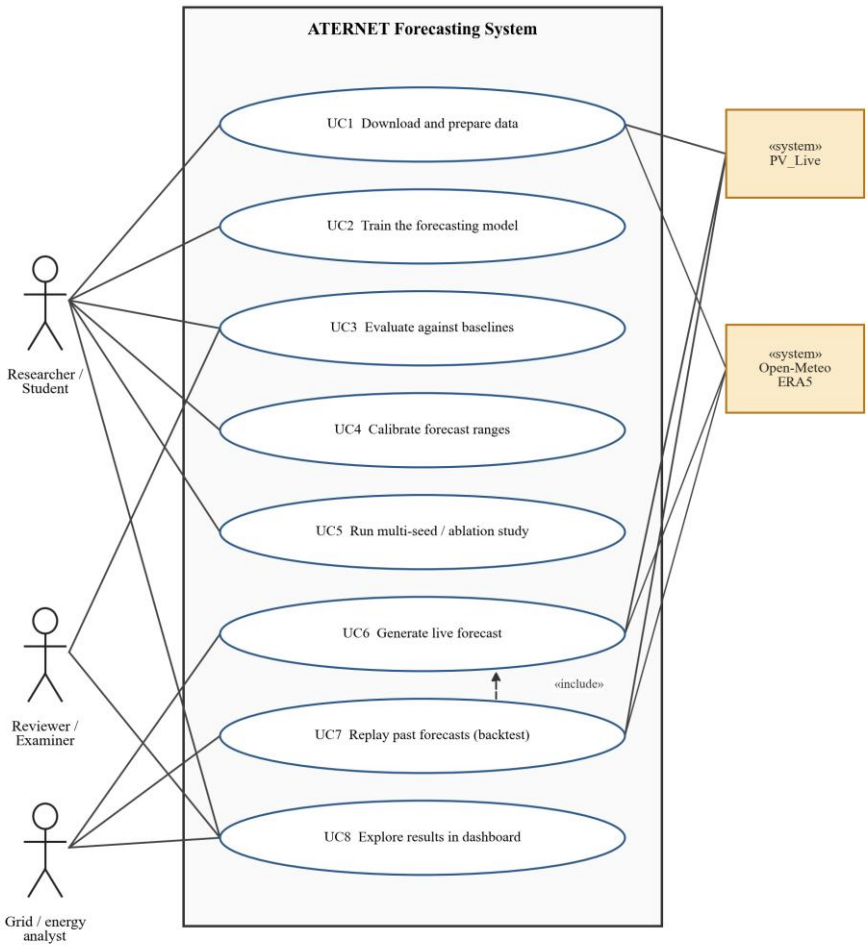


Figure 4.5: Use case diagram of the ATERNET forecasting system.

Table 4.1: Use cases and their actors.

Use case	Actor(s)	Summary
UC1 Download and prepare data	Researcher; PV_Live, Open-Meteo	Fetch PV and weather, clean, build windows and scalers
UC2 Train the model	Researcher	Joint training with early stopping and weight averaging
UC3 Evaluate against baselines	Researcher, Reviewer	Metrics for ATERNET, LightGBM, PatchTST, LSTM
UC4 Calibrate forecast ranges	Researcher	Per-regime conformal adjustment on daytime validation points
UC5 Run multi-seed / ablation study	Researcher	Repeat training over seeds, switch off model parts
UC6 Generate live forecast	Analyst; PV_Live, Open-Meteo	Forecast the next 8 hours from the latest data
UC7 Replay past forecasts	Analyst	Backtest against measured output (includes UC6)
UC8 Explore results in dashboard	Researcher, Reviewer, Analyst	Interactive pages with plain-language explanations

4.2.3 Class Diagram

Figure 4.6 shows the main classes. ATERNET is composed of the backbone, the expert pool, the router and the decoder; the expert pool aggregates four concrete experts that inherit from

an abstract ExpertBase (which defines the primary loss, the physical-prior auxiliary loss and the combined loss); the router is composed of the uncertainty-fusion module and the gating network. The Trainer uses the dataset to train ATERNET, and the Forecaster serves it for live and backtest forecasts.

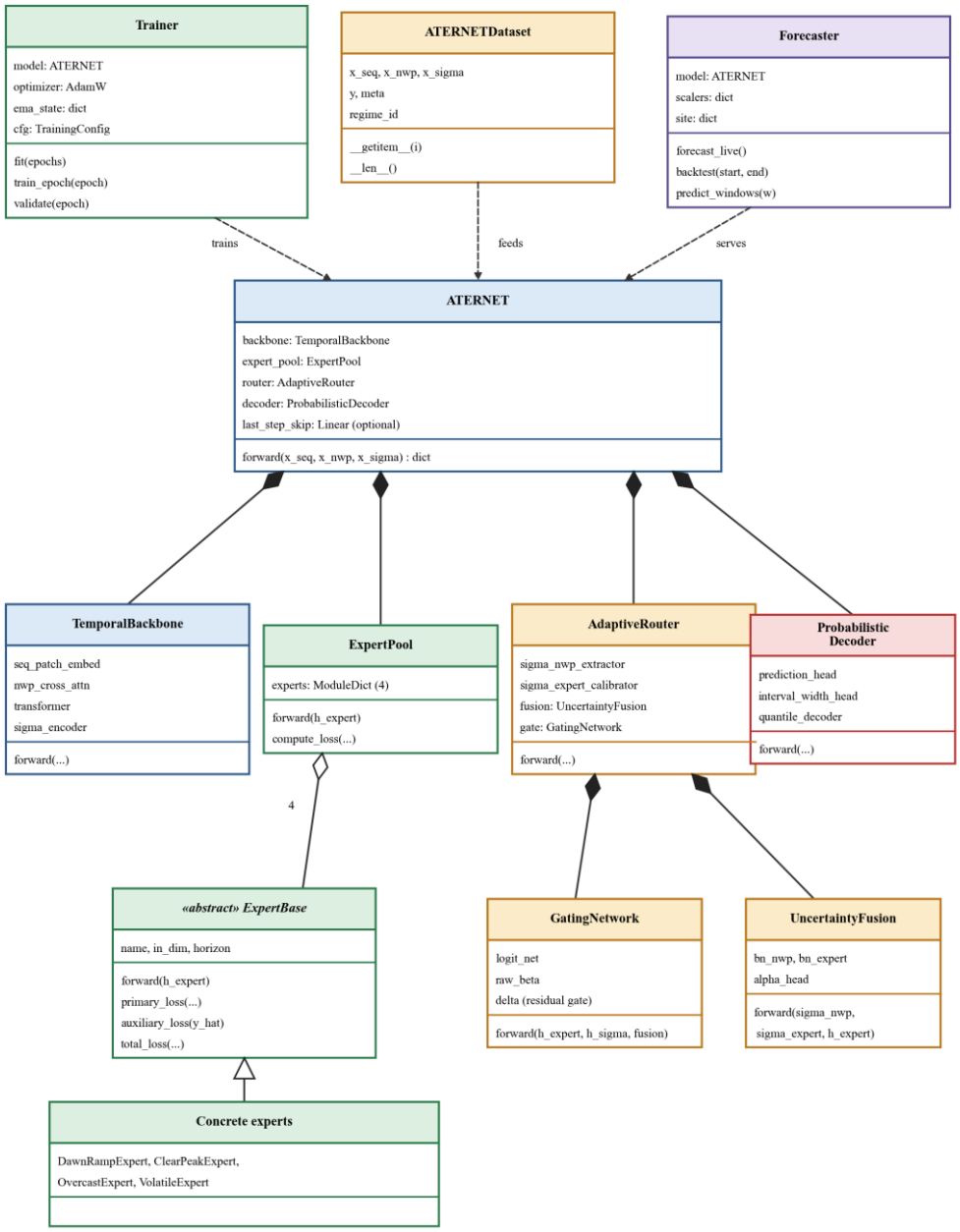


Figure 4.6: Class diagram of the main software components.

4.2.4 Sequence Diagram

Figure 4.7 shows the sequence for a live forecast: the Forecaster obtains the latest PV and weather from the two services, aligns them to the 30-minute grid, builds and scales one window, calls the model and returns a table of P10, P50 and P90 in megawatts.

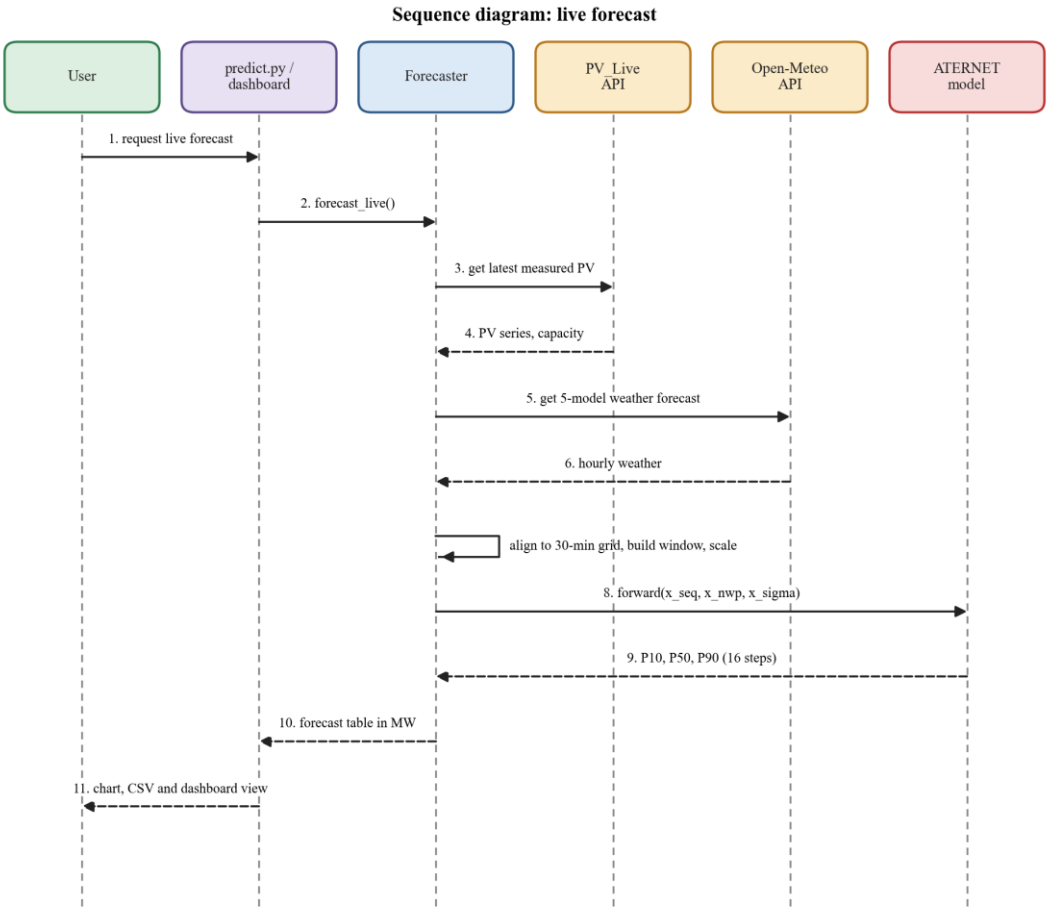


Figure 4.7: Sequence diagram of a live forecast request.

Figure 4.8 shows the sequence for training and evaluation. For every epoch the Trainer runs forward and backward passes, updates the optimiser and the averaged weights, validates with the averaged weights and keeps the best checkpoint; evaluation and calibration are then run on the saved model.

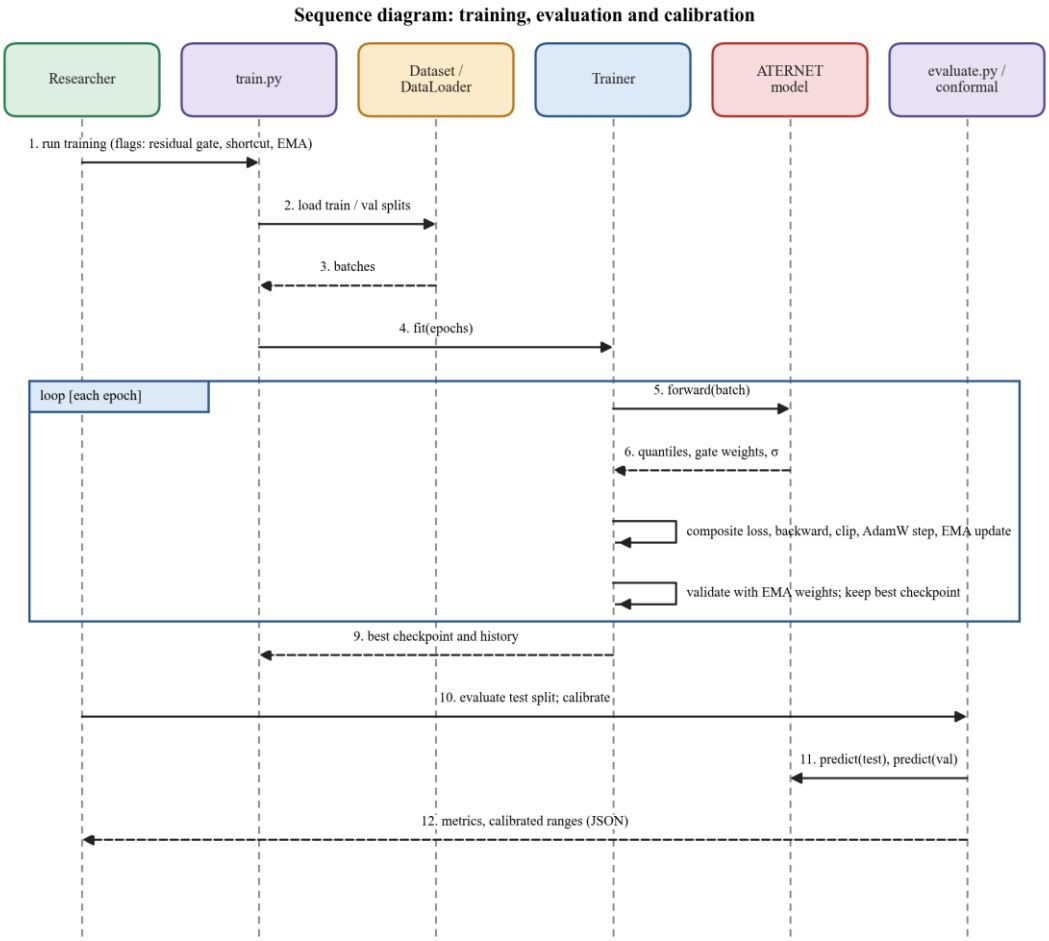


Figure 4.8: Sequence diagram of training, evaluation and calibration.

CHAPTER 5
METHODOLOGY AND TESTING

5.1 Module Description

5.1.1 Data Acquisition and Preparation (Phase 1)

Data sources. Measured regional PV generation comes from Sheffield Solar PV_Live [35], which reports half-hourly generation and installed capacity for each distribution-network region of Great Britain. Weather comes from the Open-Meteo forecast archive [36], which stores the forecasts that five independent NWP systems actually issued. We treat each system as one ensemble member, so the spread between them is a genuine measure of forecast uncertainty. Weather is sampled at three points across a region and averaged. The same models are available from the live forecast API, so the inputs seen in training and in live use have the same form.

Table 5.1: Datasets used for the two measured regions.

Item	Yorkshire	South West England
PV_Live region	23 (NPG Yorkshire)	22 (WPD South West)
Installed capacity (end of period)	1,165 MWp	2,693 MWp
Period	1 Jan 2023 – 30 Sep 2026	1 Jan 2023 – 30 Sep 2026
Half-hours / windows	65,713 / 16,413	65,713 / 16,413
Train / validation / test windows	11,489 / 2,462 / 2,462	11,489 / 2,462 / 2,462
Train period	1 Jan 2023 – 15 Aug 2025	same grid
Validation period	15 Aug 2025 – 8 Mar 2026	same grid
Test period	8 Mar – 29 Sep 2026	same grid
Weather members	GFS, ICON, UKMO, Météo-France, GEM	same

Cleaning. The generation values are converted to power and normalised by the installed capacity (which grows over time), clipped to [0, 1], checked for gaps, and the solar elevation is computed for every half-hour with the NOAA formulation, including longitude and the equation of time.

Features (Table 5.2) include the clear-sky index (measured or forecast irradiance divided by the clear-sky value), time-of-day and day-of-year encodings, the ensemble mean of each weather variable, and the spread between the weather models.

Table 5.2: Input and target tensors built in Phase 1.

Group	Shape	Contents
X_seq	48 × 10	pv_norm, ghi, cloud cover, temperature, wind speed, clear-sky index, hour sin/cos, day-of-year sin/cos
X_sigma	48 × 6	σ_NWP for irradiance, cloud, temperature and wind; a placeholder for σ_expert; normalised ensemble spread
X_nwp	64 × 4	Ensemble-mean forecast of irradiance, cloud, temperature, wind over the past 24 h and the next 8 h
y	16	Capacity-normalised PV output for the next 8 hours (30-minute steps)
meta	9	Regime id and one-hot, ramp-rate and rolling volatility statistics (used for stratification and evaluation)

Weather regimes. For stratified evaluation and for the experts, each window is tagged by simple rules at the issue time: dawn ramp (solar elevation between 2° and 20° with a clear ramp rate), clear peak (clear-sky index above 0.75 and low one-hour volatility), overcast (clear-sky index at most 0.4 and low volatility) and volatile (one-hour standard deviation of at least 0.05). Windows issued at night default to the overcast tag, which is why that regime is the largest in the test set (Table 7.8).

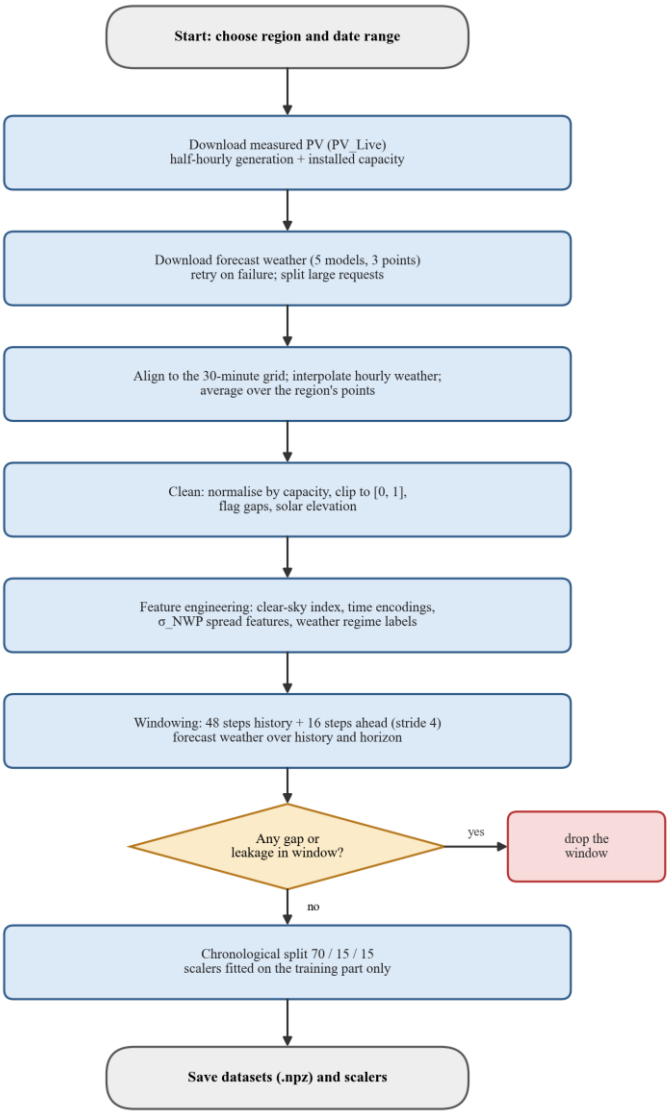


Figure 5.1: Flow of data acquisition and preparation.

Windows and splits. Each sample is 48 half-hours of history and 16 half-hours of future, taken every 4 steps (two hours). The data are split chronologically, 70% for training, 15% for validation and 15% for testing, with no shuffling, and the min-max scalers are fitted on the training part only so no information from the future reaches the model. Figure 5.2 shows one week of measured output and the corresponding forecast irradiance with the spread between the five weather models.

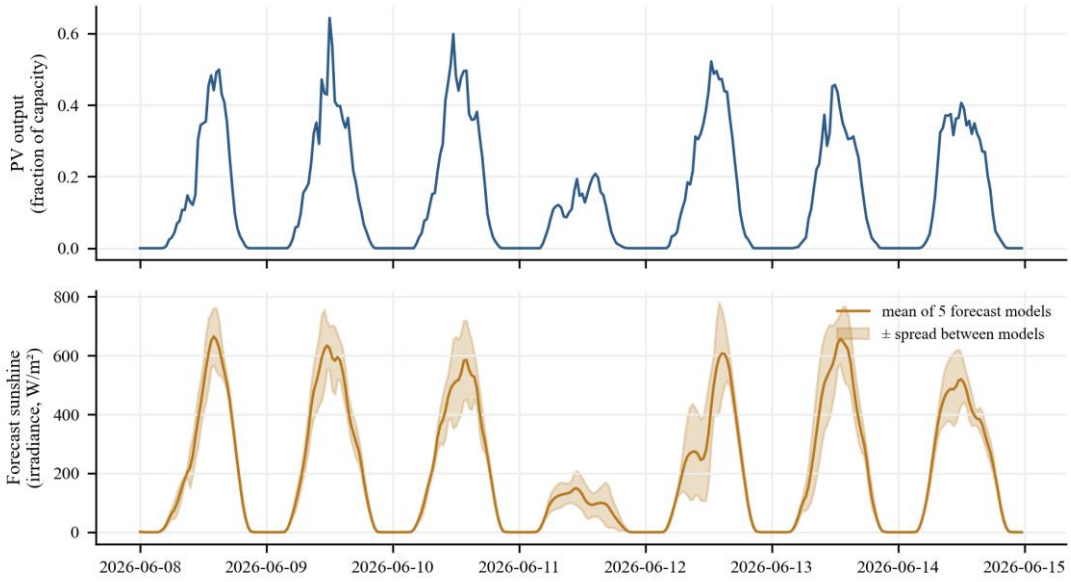


Figure 5.2: One week (8–14 June 2026) of measured PV output in Yorkshire and the ensemble forecast of irradiance with the spread between the models.

5.1.2 Temporal Backbone

The backbone follows the PatchTST design [28]. A patch length of 8 steps with a stride of 4 turns the 48-step history into 11 patches that are projected to 128 dimensions and given learnable positions. A three-layer, pre-norm Transformer encoder with eight heads and a feed-forward size of 256 processes the patches. The weather forecast is cut into patches in the same way and attended to by the PV patches. The forecast-spread features, which have a physical meaning of their own, are encoded by a separate small network and are deliberately kept out of the Transformer.

5.1.3 Expert Pool

The four experts are sized to the difficulty of their weather regime (Table 5.3). All four are run on every sample, and every expert also receives its own training loss on every sample (the dense expert loss), with the samples of its own regime weighted three times higher. This keeps all experts learning even when the gate gives them little weight.

Table 5.3: The four experts.

Expert	Hidden sizes	Loss	Built-in constraint	Parameters
Dawn ramp	256→256→128	MSE + slope hinge	ramp as residual + cumulative sum	134,929
Clear peak	128→64	MSE	none	26,064
Overcast	128→128	Huber (δ = 0.05)	output capped at 0.80	35,344
Volatile	512→256→128	MSE	none; widest, most dropout	233,872
Pool total				430,209

5.1.4 Adaptive Router

σ_NWP is the mean of the five forecast-spread features at the issue time and is not learned. σ_expert is passed through a single positive scale so its ranking is preserved. Both are variance-stabilised (log1p), batch-normalised and mixed with the learned weight α (Equation 4.1). The gate is a small network over the embeddings and uncertainty signals that returns one

score per expert; the scores are divided by the temperature and, in the final model, scaled by the residual factor δ (Equation 4.2).

5.1.5 Probabilistic Decoder

The prediction head adds a bounded correction (at most ±0.1 after a tanh) to the routed forecast to give the median. The interval-width head outputs separate lower and upper half-widths for every step through a softplus, with a learned sensitivity to σ_total. P10 and P90 are the median minus the lower and plus the upper width, clipped to the physical range [0, 1].

5.1.6 Training Procedure

All phases are trained jointly with one optimiser, because the usefulness of the router depends on the experts and vice versa. The loss is the weighted sum

$$L = L_pinball + 0.05 \cdot L_sharp + w(t) \cdot \Sigma_k L_expert,k + 0.01 \cdot L_balance \qquad (5.1)$$

The pinball loss at a fixed level τ is a proper scoring rule [4, 5]; L_sharp is the mean interval half-width, which stops the model from reaching good coverage simply by drawing very wide intervals; L_expert is the dense loss of each expert with its physical-prior term (weight 0.1); and L_balance is a small penalty against the gate collapsing onto one expert. The dense-loss weight w(t) is 1.0 for the first five epochs, decreases linearly to 0.3 over the next five and then stays at 0.3, so that the routed objective dominates once the experts are competent.

Table 5.4: Training configuration of the final model.

Setting	Value
Optimiser	AdamW [38], learning rate 1e-3, weight decay 1e-4
Learning-rate schedule	Cosine annealing to 1% of the initial rate
Batch size / epochs	64 / at most 50
Gradient clipping	Global norm 1.0
Early stopping	Patience 8 epochs on the validation pinball loss
Model selection	Best validation pinball loss (not the composite loss, whose weights change)
Weight averaging	EMA with decay 0.999 and warm-up min(0.999, (1+t)/(10+t)) [32]; validation and checkpoints use the averaged weights
Window / horizon / stride	48 / 16 / 4 half-hours
Backbone	Patch 8, stride 4, d_model 128, 8 heads, 3 layers, feed-forward 256, dropout 0.1
Random seed	42 for the main model; 42, 43, 44 for multi-seed studies

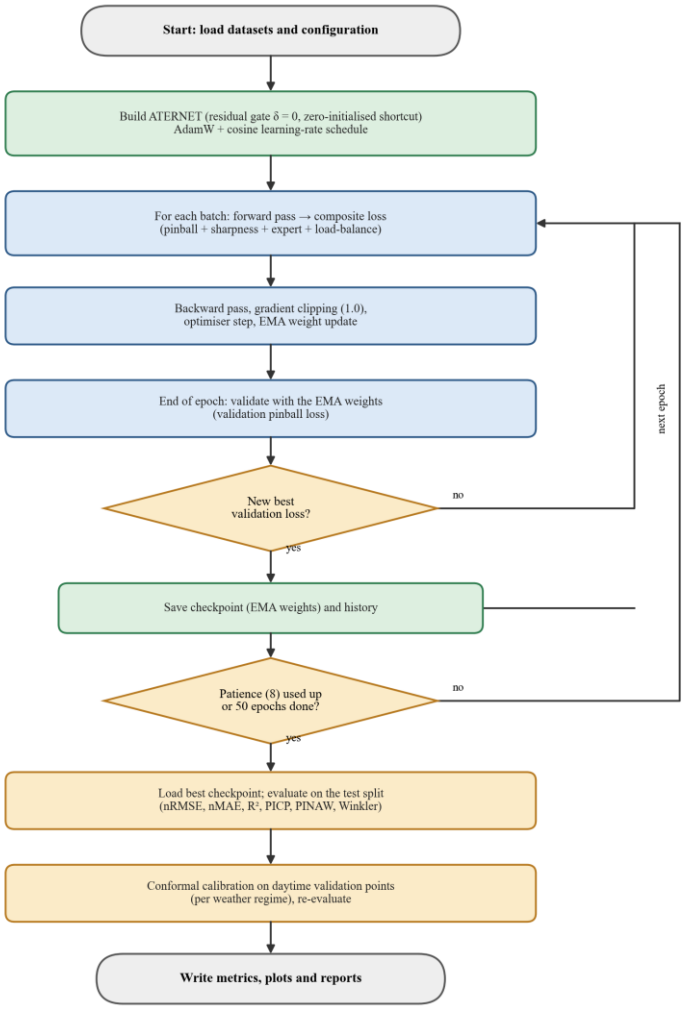


Figure 5.3: Flow of training, evaluation and calibration.

Exponential moving average. After every optimiser step a smoothed copy of the weights is updated as $\theta_{ema} \leftarrow d \cdot \theta_{ema} + (1 - d) \cdot \theta$, with a warm-up that starts the decay low so the average does not remain close to the random initial weights. Validation, checkpointing and prediction use the smoothed weights, which proved to be one of the most effective changes in this work (Section 7.4).

5.1.7 Interval Calibration

The raw P10–P90 range of a model is not guaranteed to hold 80% of the observations. We use conformalized quantile regression [18]. For each weather regime and each daytime point of the validation set we compute the conformity score

$$E = max(P10 - y , y - P90) \tag{5.2}$$

and take its finite-sample quantile at level $\lceil (n + 1)(1 - \alpha) \rceil / n$ with $\alpha = 0.20$. The resulting value is subtracted from P10 and added to P90 on daytime points of the test set, and the interval is kept non-crossing and inside [0, 1]. Two details matter. First, scores are computed for each forecast step, not averaged over the 16 steps of a window, because an average of signed violations is close to zero or negative and produces no adjustment. Second, only daytime points are used and adjusted, where daytime means the forecast irradiance input

exceeds a small threshold; at night the truth and the interval are both zero and are trivially 'covered', which would otherwise inflate every coverage figure. Both were defects of our first implementation (Table 5.8).

5.1.8 Baseline Models

Table 5.5: Baseline models used for comparison.

Model	Description
Persistence	The next hours equal the last observed value; used for skill scores
LightGBM [29]	One gradient-boosted model per quantile and per horizon step (48 models), 63 leaves, learning rate 0.05, up to 500 rounds with early stopping; 93 input features: latest step, forecast weather for the horizon, forecast-spread and metadata
PatchTST [28]	Transformer encoder on patches with three quantile outputs; no experts or routing
LSTM [26]	Two layers of 128 units, dropout 0.1, concatenated inputs, three quantile heads, early stopping with patience 10

All baselines read exactly the same windows, splits and scalers as ATERNET, so they receive the same forecast weather, including the next eight hours. They were run with the default settings listed above and were not specially tuned, a limitation we discuss in Chapter 7.

5.1.9 Evaluation Metrics

Table 5.6: Evaluation metrics.

Metric	Meaning	Better when
nRMSE (%)	Root-mean-square error as a percentage of installed capacity	lower
nMAE (%)	Mean absolute error as a percentage of capacity	lower
R^2	Share of the variance of the observations explained by the P50 forecast	higher
Skill vs persistence	1 − RMSE / RMSE of the persistence forecast	higher
PICP	Share of observations inside [P10, P90]; the target is 0.80	close to 0.80
PINAW	Mean interval width as a fraction of capacity	lower (at equal coverage)
Winkler score	Mean of width + (2/α) × (distance outside the interval), α = 0.2 [30]	lower
Pinball loss	Mean quantile loss at levels 0.1, 0.5, 0.9	lower

PICP is reported in two ways: over all points, as is common in the literature, and over daytime points only. Winkler and the error metrics are computed over all points for every model in the same way, so they are comparable between models. Differences between models are called significant only when they exceed about twice the run-to-run standard deviation.

5.1.10 Inference and Dashboard

The Forecaster class builds a window from the latest data, scales it with the saved scalers and calls the model; the backtest command replays past issue times and scores them against what was measured. The dashboard (Chapter 6) is a Streamlit application that reads the saved predictions, the multi-seed summary and the calibration results, and re-computes the built-in safety checks live on the model.

5.1.11 Second Region and Simulated Site

A second measured region and a simulated site were added by configuration only.

Environment variables choose the region and move the data, checkpoints and outputs to a separate workspace so the default experiments cannot be overwritten. For VIT Vellore, where no measured record exists, the PV output is simulated by a documented physical model (temperature derating, soiling, inverter clipping and about 1% noise) driven by ERA5 reanalysis weather [37], while the model's inputs remain the forecast models. Using a different weather source for the 'truth' and for the inputs ensures that the forecast-versus-actual error is real and that the task is not simply inverting a formula applied to the inputs.

5.2 Testing

5.2.1 Automated Tests

The project has 74 automated tests (Table 5.7) that run in about fifteen seconds. They check the data pipeline (shapes, absence of NaNs, a temporal-leakage check, training-only scaler fitting), the architectural guarantees, the loss functions and metrics against hand-computed values, the real-time data parsing and window layout, the new model options, the conformal scores, the network retry logic and the simulated-PV model.

Table 5.7: Automated test suite (74 tests, all passing).

Test file	Tests	Covers
test_phase1_pipeline.py	9	Window shapes, no NaNs, leakage check, scaler fitting, regime coverage, gap rejection
test_phases_2_5.py	19	Three structural guarantees, output ranges, σ_expert detachment, gradient health
test_losses_and_metrics.py	14	Hand-computed losses and metrics, pinball asymmetry, Winkler trade-off
test_integration.py	8	Real training run, loss decrease, gate non-collapse, checkpoint round-trip
test_realtime_sources.py	5	Solar geometry, weather parsing and alignment, live window layout
test_residual_gate.py	3	Gate starts exactly uniform; δ receives gradient; default model unchanged
test_last_step_skip.py	3	Shortcut is a no-op at start and receives gradient
test_accuracy_options.py	3	Metadata inputs, extra loss term, option validation
test_conformal_scores.py	2	Per-step scores restore coverage where window averages do not
test_http_retry.py	3	Truncated downloads are retried, large requests split, 4xx not retried
test_simulated_pv.py	5	No output at night, clipping, temperature effect, soiling, region flagged
Total	74	

5.2.2 Experimental Protocol

Every reported model was trained from several random starts (seeds 42, 43 and 44; the LSTM also 45, 46 and 47 because of its instability). We report the mean and the sample standard deviation. The test set is never used for model selection: early stopping and calibration use the validation set. Hyper-parameters were chosen on Yorkshire, and the second region was run afterwards without any change.

5.2.3 Validity Checks

- **No look-ahead.** Only measured values up to the issue time are inputs; the weather forecasts for the next eight hours are inputs because they would be available at that time in live use.
- **Chronological splitting** with no shuffling and scalers fitted on training data only.
- **Reproduction check.** The LightGBM baseline run through the multi-seed tool with seed 42 reproduced the earlier single-run result exactly, confirming that the tool does not change the baselines.
- **Out-of-sample backtest** on 1–16 October 2026, after the end of the training period, using only data available at each issue time (Section 7.7).

5.2.4 Defects Found and Corrected

Testing and analysis revealed several defects and wrong conclusions in earlier versions of the work. They are listed in Table 5.8 because correcting them changed the results and the conclusions of this report.

Table 5.8: Defects found during testing and how they were corrected.

Finding	Consequence	Correction
Early data source mis-scaled PV units and capacity	Peaks clipped; unrealistic targets	Convert Wh to kW, use the real capacity per row
Weather input stopped at the issue time	Model never saw the forecast for the hours it predicts	Weather now spans history plus the 8-hour horizon
Solar elevation ignored longitude and the equation of time	Several hours of error away from Greenwich	NOAA formulation in src/utils/solar.py
Two ablations were mis-implemented	One crashed; one did not disable what it claimed	Fixed and re-run
Learned gate raised error and variance	nRMSE 4.39 ± 0.20 against 4.12 for uniform averaging	Residual gate starting at uniform routing
Single-seed comparisons	An LSTM run gave 9.19% where other seeds gave 3.7%	All comparisons use several seeds
Calibration used a window average of signed violations	Adjustments near zero or negative; volatile regime got narrower	Per-step scores
Night points counted in coverage	Coverage looked like 0.80 while daytime coverage was 0.67	Daytime-only calibration and reporting
Truncated downloads on this network	Pipeline crashed on corrupted JSON	Retry plus automatic request splitting
Dashboard demo ignored the shortcut; routing text was inaccurate	Misleading demonstration	Demo recomputes the full path; wording corrected

CHAPTER 6
PROJECT DEMONSTRATION

The results of the project are presented in an interactive web dashboard written with Streamlit [34]. It is started with the single command **streamlit run app.py** and opens in a browser at http://localhost:8501. The left menu has nine pages; each opens with a short blue box that says in plain language what the page shows, and a collapsible glossary defines every technical term (typical error, range hit-rate, range quality score and so on). All screenshots in this chapter were captured from the running application.

Table 6.1: Pages of the dashboard.

Page	What it shows
Overview	Headline numbers, a colour-coded scoreboard against other models, an example forecast and results by weather situation
Is ATERNET better?	The multi-seed comparison for two regions, charts, calibration and what each model part contributes
India demo (simulated)	VIT Vellore forecasts on a clearly marked simulated plant, with a forecast viewer
Live forecast	Forecast of the next 8 hours from the latest data and replay of past forecasts
Explore forecasts	Browse individual test cases by weather situation
How the experts work	How much the model trusts each of the four experts
How sure is the model?	Relation between the model's unsureness, the range width and the real error
Built-in safety checks	Live demonstration of the three structural guarantees
Training history	Learning curves, expert usage during training, learning rate

6.1 Overview Page

The overview (Figure 6.1 to 6.2) opens with five headline numbers, each with a tooltip that gives the technical name. Below them, a green or yellow summary and a scoreboard compare ATERNET with LightGBM, PatchTST and LSTM for both regions. A pill is green when ATERNET is better, yellow when the result is about the same, red when it is worse and grey when the other model is unreliable. The first part of the figure also shows the navigation menu.

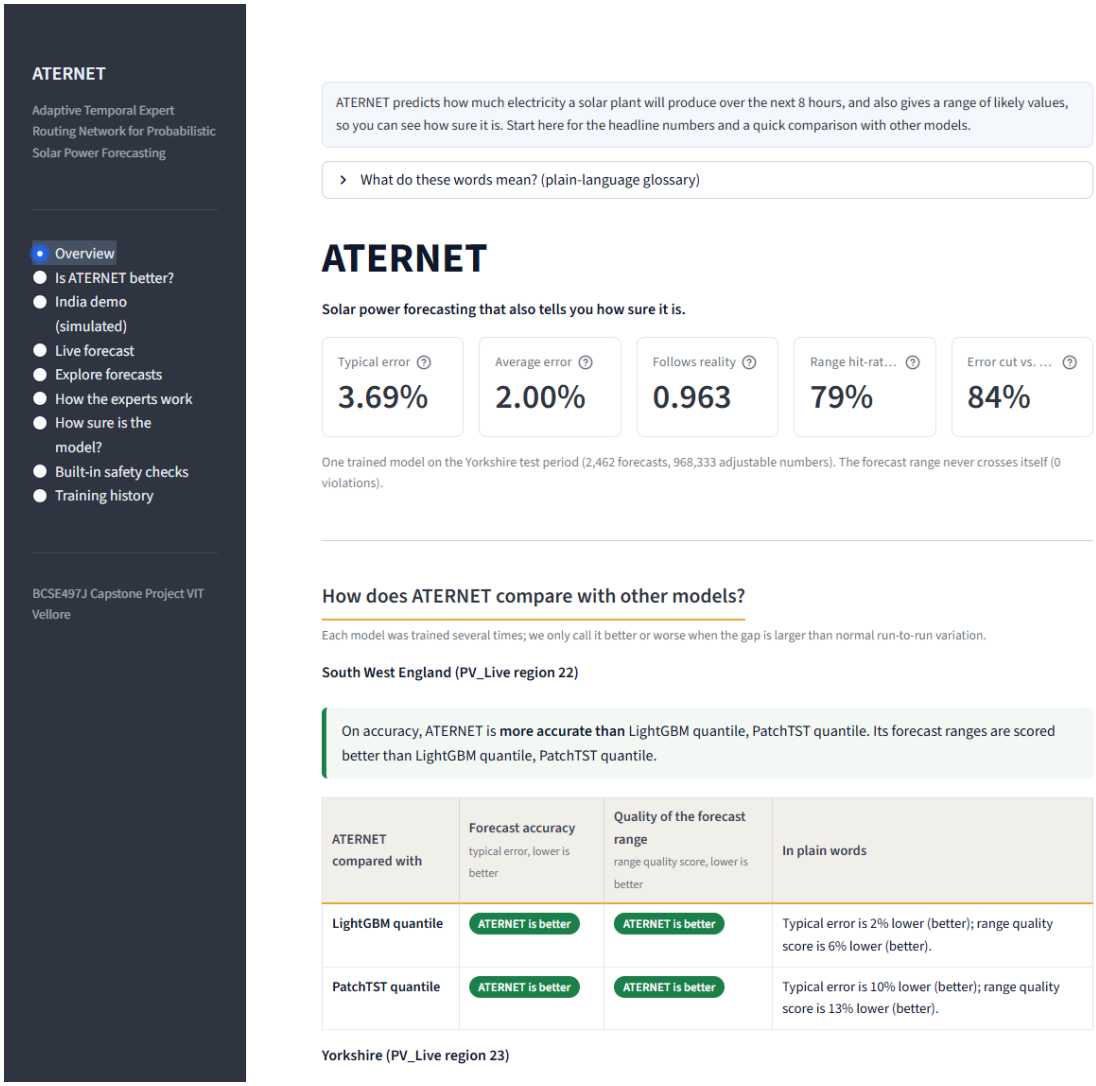


Figure 6.1: Overview page of the dashboard (screenshot) (part 1 of 2)



Figure 6.2: Overview page of the dashboard (screenshot) (part 2 of 2)

6.2 Is ATERNET Better?

This page (Figure 6.3 to 6.5) is the key one for evaluation. A region is chosen at the top; the short answer, the scoreboard and two bar charts follow. A difference is called 'better' or 'worse' only when it exceeds about twice the run-to-run variation; otherwise it is 'about the same'. Further down, the page shows whether the forecast range holds what it promises and which parts of the model matter.

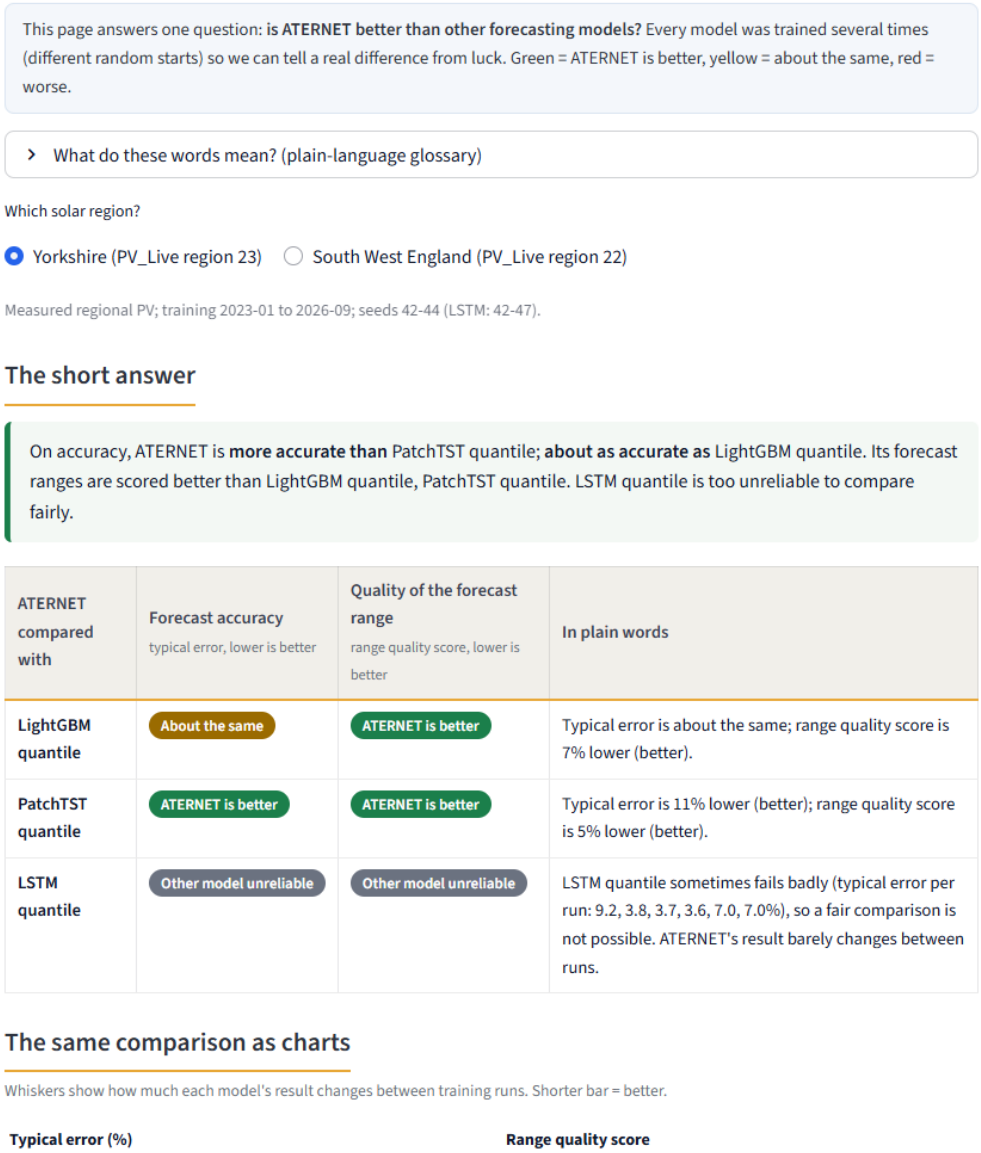
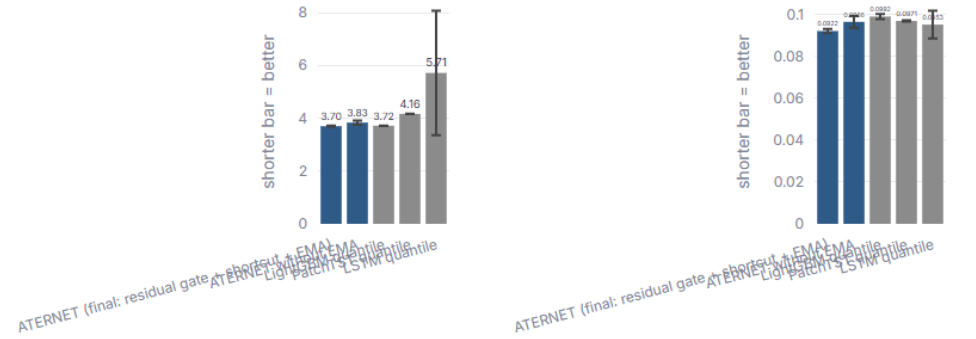


Figure 6.3: Comparison page for the Yorkshire region (screenshot) (part 1 of 3)



› Show the detailed numbers

Does the forecast range hold what it promises?

The range is meant to contain the real output 80% of the time. Checked in daytime only.

Before calibration	After calibration	Target
67%	79%	80%
	↑ +12 points	

Calibration was applied to ATERNET only. The comparison tables above use everyone's raw ranges, so they are fair.

Which parts of ATERNET actually help?

We switched off one part at a time and checked whether results got worse.

Figure 6.4: Comparison page for the Yorkshire region (screenshot) (part 2 of 3)

Part removed	Typical error change	Range quality change	Verdict
Without the extra training signal given to every expert	+0.111 points	+0.0082	Removing it mak
Without the gate (all experts always trusted equally)	+0.030 points	-0.0014	No clear differen
Without the 'spread votes when unsure' control	-0.022 points	-0.0003	No clear differen
Without letting the range width follow the model's unsureness	+0.024 points	-0.0003	No clear differen
Using only the weather-forecast disagreement as the unsureness signal	-0.047 points	-0.0019	No clear differen
Using only the experts' disagreement as the unsureness signal	-0.001 points	-0.0010	No clear differen
Using one expert instead of four	+0.040 points	+0.0015	No clear differen
Without extra training weight on each expert's own weather type	-0.056 points	-0.0013	No clear differen

Only one change clearly mattered: **Without the extra training signal given to every expert**. The other parts made no measurable difference on this data, so we do not claim they improve accuracy.

› What these results do and do not show

Figure 6.5: Comparison page for the Yorkshire region (screenshot) (part 3 of 3)

6.3 India Demonstration (Simulated)

The India page (Figure 6.6 to 6.7) begins with a red banner stating that the data are simulated

and explaining how. It then shows the results for the simulated VIT Vellore plant, a scoreboard against LightGBM, an interactive forecast viewer in which the test case is chosen with a slider, and the diagnostic plots.

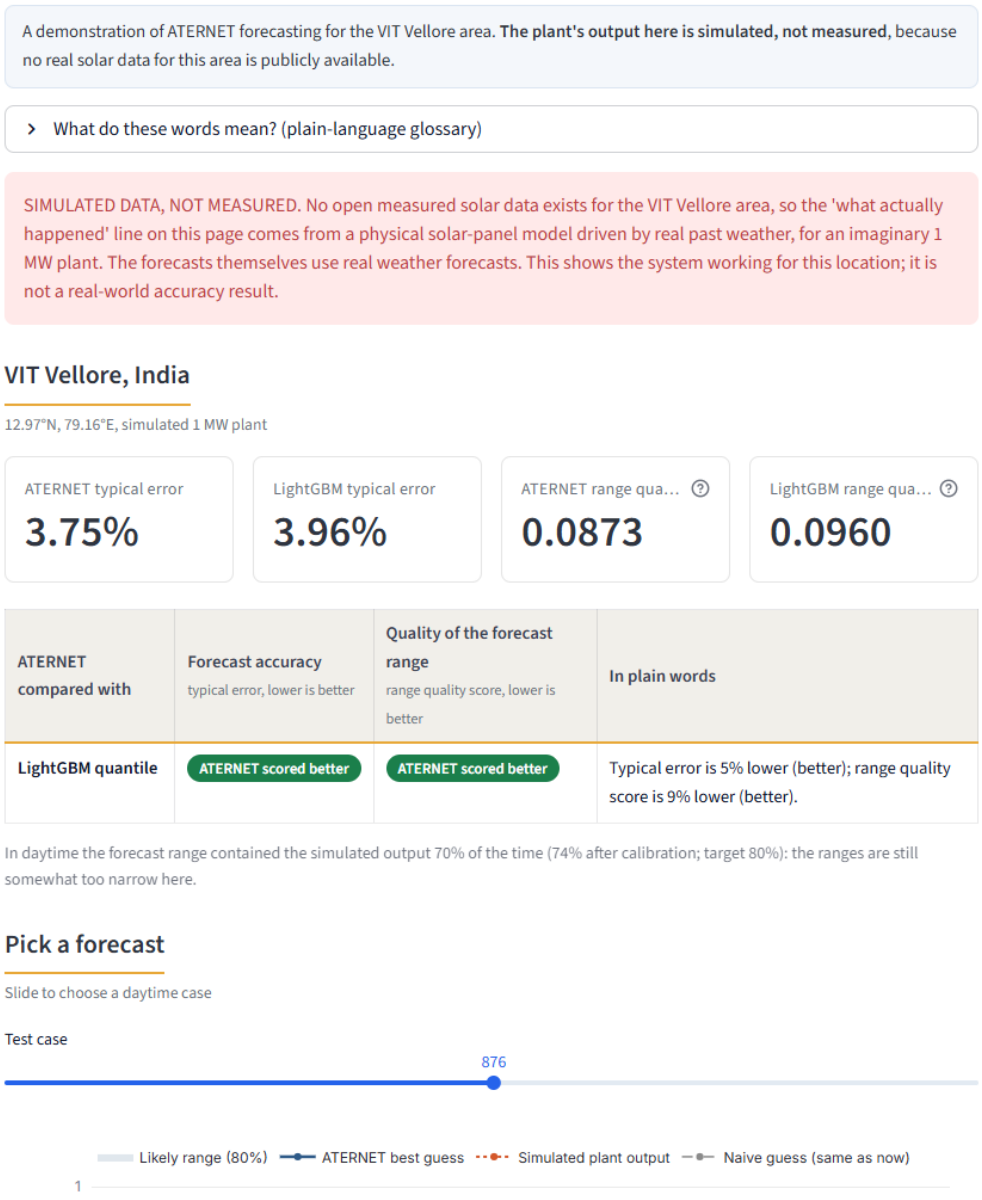


Figure 6.6: India demonstration page with the simulated-data warning (screenshot) (part 1 of 2)

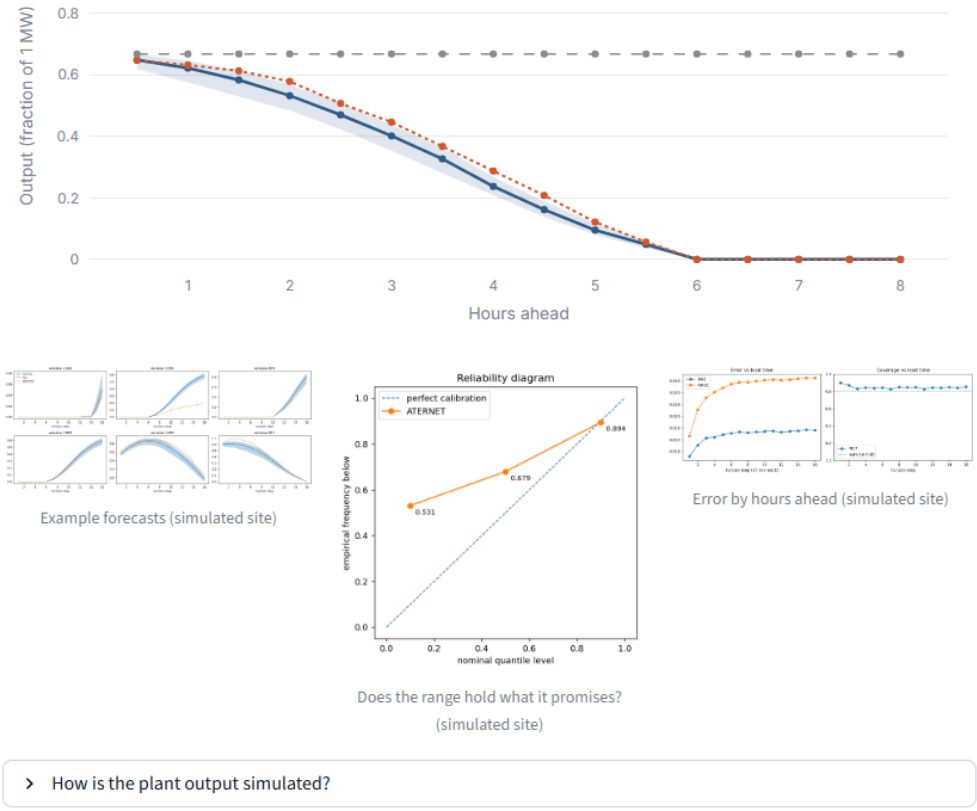


Figure 6.7: India demonstration page with the simulated-data warning (screenshot) (part 2 of 2)

6.4 Uncertainty, Safety Checks and Further Pages

Figure 6.8 shows the page on which the model's 'unsureness' score and the width of its forecast range are compared with the real error of each forecast, together with the calibration table. Figure 6.9 shows the safety checks: moving the slider changes the unsureness fed to the decoder and gate, and the page reports that the range never flips (zero violations), widens monotonically and that the experts' votes spread out.

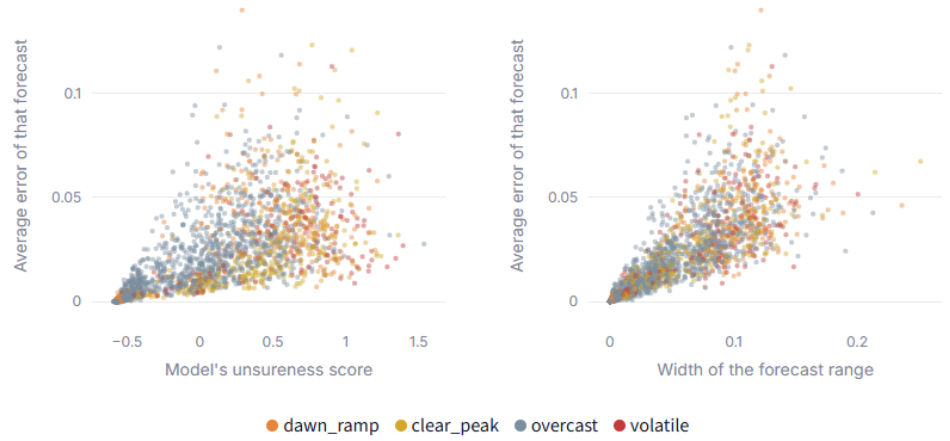
A good forecast also says how sure it is. Here we check that the model's 'unsureness' really goes up when its errors are bigger, and that its forecast ranges contain the real value about as often as promised.

› What do these words mean? (plain-language glossary)

How sure is the model?

The model gives every forecast an 'unsureness' score and draws a wider range when it is less sure. Below we check that this score really goes up when the errors are bigger.

Does 'unsure' predict bigger errors?	Do wider ranges go with bigger err…	Weather-vs-model mix (α)
0.688	0.815	0.157



Conformal calibration

Per-regime conformal calibration fitted on daytime validation points

Regime	n	Daytime PICP before	Daytime PICP after	PINAW before	PINAW after
dawn_ramp	352	0.658	0.79	0.0532	0.0655
clear_peak	459	0.672	0.828	0.0599	0.0792
overcast	1446	0.669	0.779	0.041	0.048
volatile	205	0.685	0.797	0.0691	0.0845

Coverage is measured on daytime points only (nominal 0.80). Night points have a true value and interval of exactly 0 and are trivially covered, which would inflate the number.

Figure 6.8: Page showing how the model's unsureness relates to the real error (screenshot)



Figure 6.9: Built-in safety checks page (screenshot)

Figure 6.10 shows the forecast explorer, where individual test cases can be browsed by weather situation, and Figure 6.11 the page on how the four experts are used.

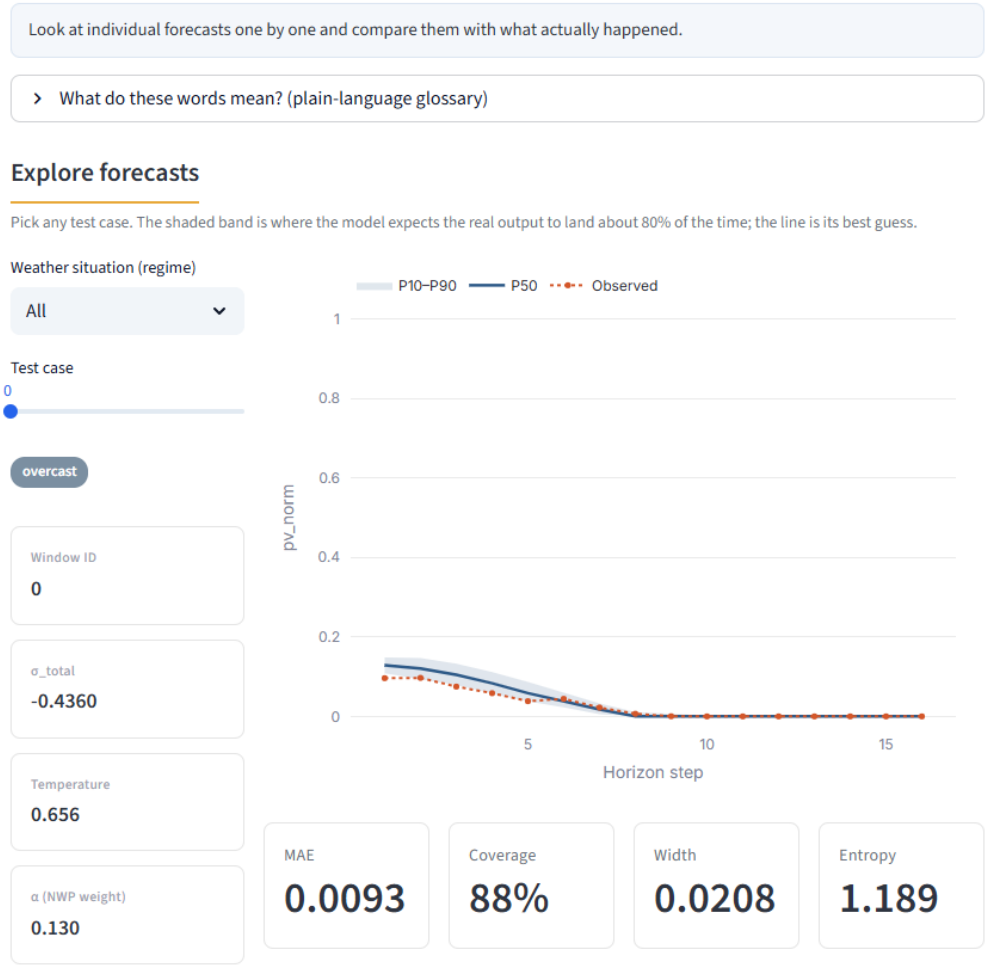


Figure 6.10: Forecast explorer page (screenshot)

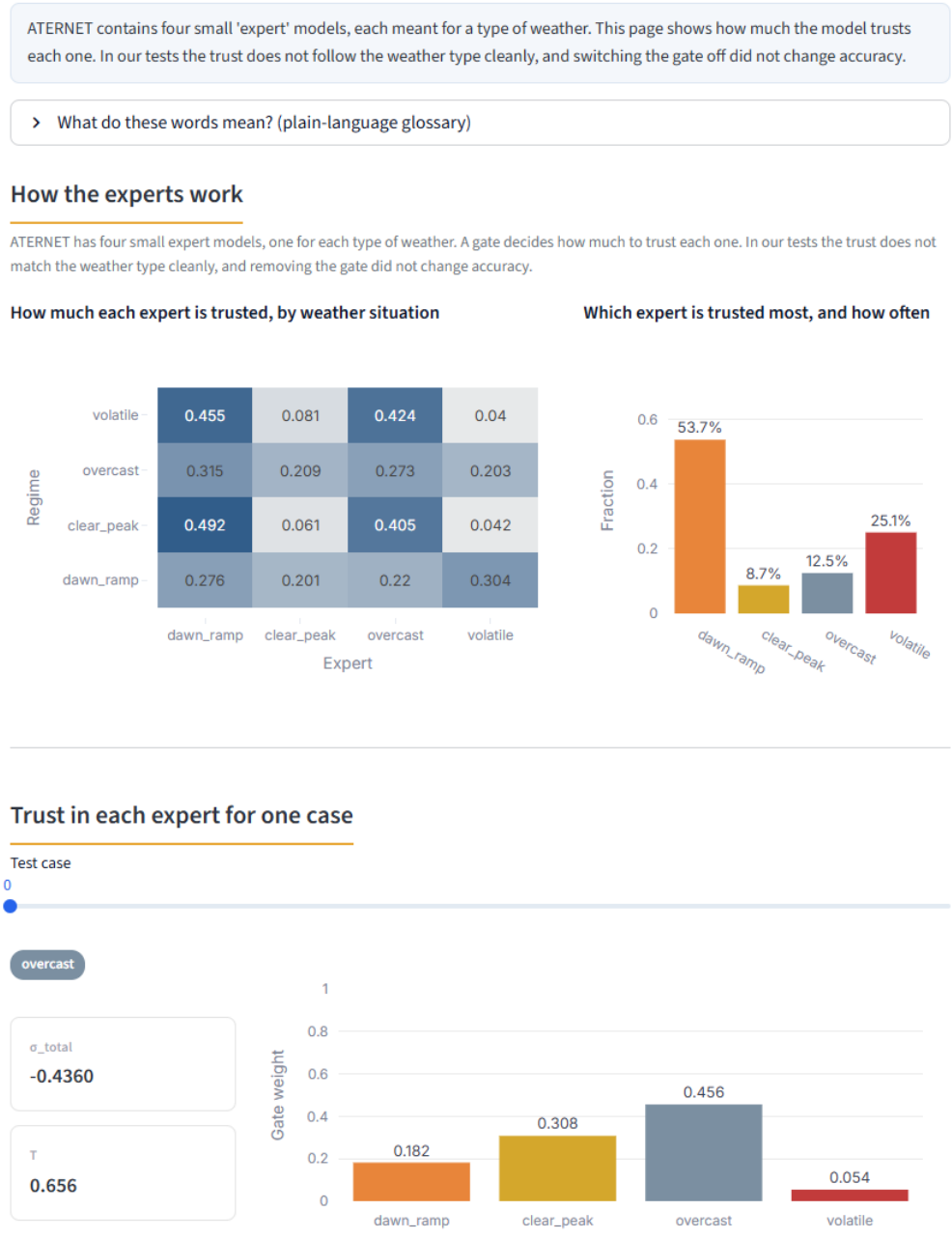


Figure 6.11: Page showing how the four experts are trusted (screenshot)

6.5 Running the Demonstration

To reproduce the demonstration from the project folder:

```
python -m pytest -q                  # 74 tests pass
python build_results_summary.py      # refresh docs/results_summary.json
streamlit run app.py                 # open http://localhost:8501
python predict.py live               # optional: forecast the next 8 hours
```

CHAPTER 7
RESULT AND DISCUSSION (COST ANALYSIS AS APPLICABLE)

7.1 Experimental Setup

All experiments use the datasets of Table 5.1, the models and settings of Chapter 5 and the evaluation metrics of Table 5.6. Unless stated otherwise, a result is the mean and standard deviation of three independent training runs (seeds 42, 43 and 44); the LSTM, whose runs differ greatly, was run six times. Every value is computed on the held-out test period (8 March to 29 September 2026), which was not used for training, early stopping or calibration. A difference between two models is called significant only if it exceeds about twice the larger run-to-run standard deviation. A single training run takes about ten minutes on the laptop described in Section 3.3.1.

7.2 Results on Yorkshire

Table 7.1 and Figure 7.1 compare ATERNET with the three baselines on the Yorkshire region.

Table 7.1: Yorkshire test results, mean ± standard deviation over training runs (lower nRMSE, nMAE, PINAW and Winkler are better). PICP counts night points; see Section 7.5.

Model	nRMSE %	nMAE %	R^2	PICP (all)	PINAW	Winkler
ATERNET (final)	3.70 ± 0.02	2.00 ± 0.01	0.9626	0.809	0.0496	0.0922 ± 0.0010
ATERNET without EMA	3.83 ± 0.08	2.10 ± 0.05	0.9598	0.820	0.0536	0.0966 ± 0.0028
LightGBM	3.72 ± 0.01	1.95 ± 0.00	0.9623	0.814	0.0644	0.0992 ± 0.0013
PatchTST	4.16 ± 0.00	2.22 ± 0.01	0.9526	0.880	0.0737	0.0971 ± 0.0003
LSTM (6 runs)	5.71 ± 2.36	2.47 ± 0.66	0.8981	0.830	0.0613	0.0953 ± 0.0067

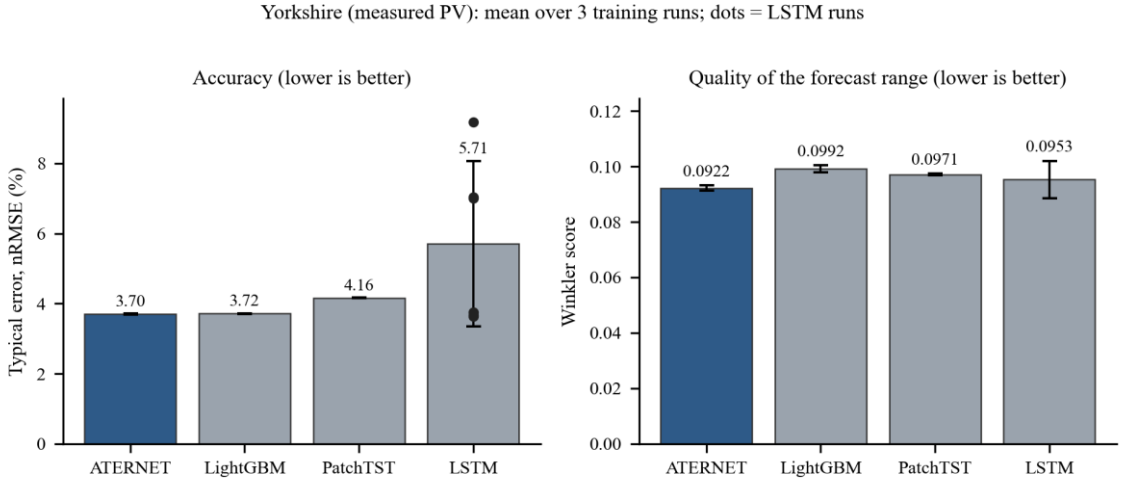


Figure 7.1: Yorkshire: accuracy and quality of the forecast range for ATERNET and the baselines.

The final ATERNET reaches an nRMSE of **3.70 ± 0.02%**. LightGBM scores 3.72 ± 0.01%; the 0.02 gap is about one standard deviation of ATERNET, so the two are **equally accurate** on this region (LightGBM has a slightly lower mean absolute error, 1.95% against 2.00%, and the R^2 values are equal). ATERNET is clearly better than PatchTST (4.16 ± 0.00%; ATERNET's error is about 11% lower). The Winkler score of ATERNET (0.0922 ± 0.0010) is lower than that of LightGBM (0.0992 ± 0.0013) and PatchTST (0.0971 ± 0.0003), and its

intervals are narrower (PINAW 0.0496 against 0.0644 and 0.0737). The narrower ranges partly reflect that ATERNET's raw ranges under-cover in daylight (Section 7.5), and the daylight coverage of the baselines was not measured, so the Winkler score, which penalises missed values, is the fairer comparison. The spread between ATERNET's three runs is very small (0.016), which also shows that training is stable.

The single main model (seed 42) used by the dashboard and the backtest reaches nRMSE 3.69%, nMAE 2.00% and R^2 0.963. Persistence (assuming the next hours equal the present) has an nRMSE of 22.5%, so ATERNET cuts the error by 84%.

The LSTM baseline is unstable. Three of its six runs ended at 7.0 to 9.2% nRMSE, and the other three at 3.63 to 3.76%. Table 7.2 lists every run. On its good runs the LSTM is about as accurate as ATERNET and two of the three had a lower Winkler score (0.086 to 0.090 against 0.092), so we do not claim that ATERNET beats a well-trained LSTM; we claim only that ATERNET delivers its accuracy on every run, which the LSTM does not. We did not investigate why the LSTM fails on some seeds.

Table 7.2: Test nRMSE (%) of every training run on Yorkshire ('–' = not run).

Seed	ATERNET	LightGBM	PatchTST	LSTM
42	3.69	3.71	4.16	9.19
43	3.68	3.72	4.17	3.76
44	3.71	3.72	4.16	3.67
45	–	–	–	3.63
46	–	–	–	7.00
47	–	–	–	7.04

7.3 Results on South West England

To test whether the result depends on Yorkshire, the whole pipeline was re-run on the PV_Live region of South West England using a separate workspace. ATERNET used exactly the same settings as in Yorkshire; the baselines used their defaults. The LSTM and the ablations were not run on this region.

Table 7.3: South West England test results over three training runs.

Model	nRMSE %	nMAE %	R^2	PICP (all)	PINAW	Winkler
ATERNET (final)	2.63 ± 0.01	1.38 ± 0.00	0.9828	0.846	0.0375	0.0634 ± 0.0004
LightGBM	2.69 ± 0.00	1.38 ± 0.00	0.9821	0.787	0.0410	0.0670 ± 0.0003
PatchTST	2.92 ± 0.03	1.55 ± 0.01	0.9787	0.902	0.0610	0.0732 ± 0.0040

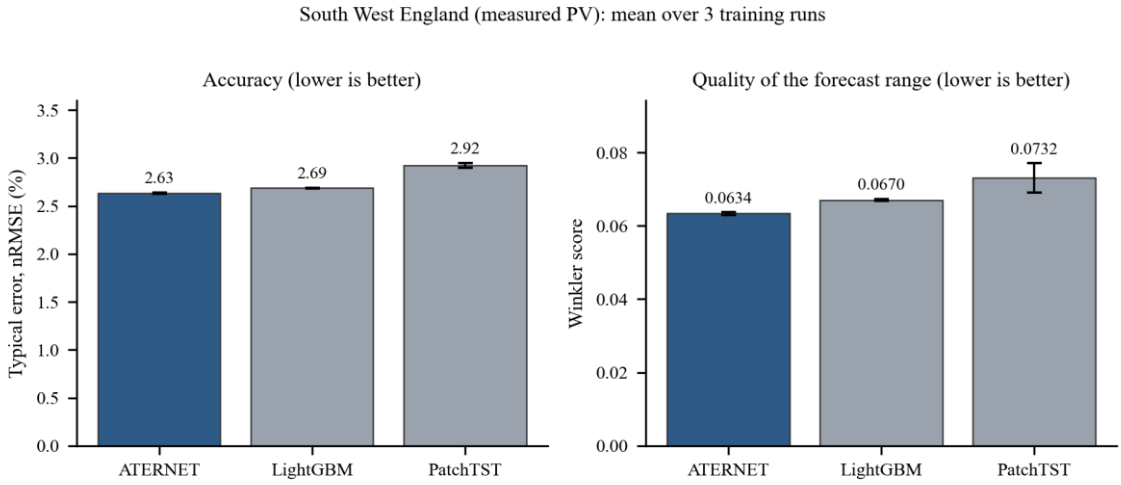


Figure 7.2: South West England: accuracy and quality of the forecast range.

Here ATERNET is more accurate than both baselines: nRMSE 2.63 ± 0.01% against 2.69 ± 0.00% for LightGBM and 2.92 ± 0.03% for PatchTST. The gap to LightGBM (about 2%) is several times the standard deviations, so it is a real but modest advantage. The Winkler score is also lower (0.0634 against 0.0670 and 0.0732). The mean absolute errors of ATERNET and LightGBM are equal (1.38% and 1.38%), so ATERNET's advantage lies in the larger errors that dominate the RMSE. Because ATERNET's settings were chosen on Yorkshire and not adjusted, this region is an out-of-sample check of the design.

7.4 Development of the Model and Ablation Studies

The final model is the result of a sequence of changes, each tested over three seeds (Table 7.4 and Figure 7.3).

Table 7.4: Development of the model on Yorkshire (three seeds each).

Configuration	Runs	nRMSE %	Winkler	PICP (all)
Original learned gate	3	4.39 ± 0.20	0.1067	0.843
Uniform average (no gate)	3	4.12 ± 0.00	0.1005	0.815
Residual gate	3	4.12 ± 0.04	0.0998	0.811
Residual gate + last-step shortcut	3	3.83 ± 0.08	0.0966	0.820
Residual gate + shortcut + EMA (final)	3	3.70 ± 0.02	0.0922	0.809

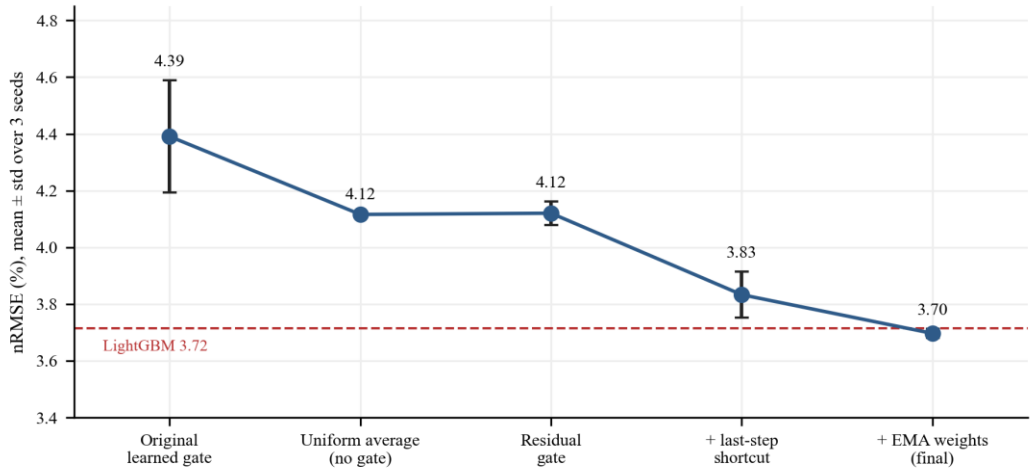


Figure 7.3: nRMSE of each stage of the development (points = mean, whiskers = standard deviation; dashed line = LightGBM).

- **The learned gate hurt.** With the original gate the error was 4.39 ± 0.20%, worse and far more variable than simply averaging the four experts (4.12 ± 0.004%). Every run with the learned gate was worse than every run with uniform averaging.
- **The residual gate removed the harm** (4.12 ± 0.04%). It starts as the uniform average and can only move away from it if that lowers the validation loss. It matched, but did not beat, uniform averaging.
- **The last-step shortcut** lowered the error to 3.83%. In the earlier model the gap to LightGBM was largest at the first forecast steps, which showed that the model was not using the latest observation well; the shortcut gives it a direct linear path from that observation.
- **Weight averaging (EMA)** lowered the error to 3.70% and reduced the run-to-run spread about five-fold.

To find out which parts of the final design contribute, we switched them off one at a time (Table 7.5 and Figure 7.4). These ablations were trained without EMA and are compared with the same configuration without EMA.

Table 7.5: Ablation studies on Yorkshire (three seeds each). Δ is the change relative to the reference; 'worse without it' means removing the part made the result worse beyond about two standard deviations.

Variant	nRMSE %	Δ nRMSE	Winkler	Δ Winkler	PICP (all)	Effect
Reference (all parts, no EMA)	3.833 ± 0.081	–	0.0966	–	0.820	–
No dense expert loss	3.944 ± 0.079	+0.111	0.1048	+0.0082	0.776	worse without it
No gate (uniform average)	3.863 ± 0.106	+0.030	0.0952	-0.0014	0.824	no clear effect
Fixed temperature	3.811 ± 0.075	-0.022	0.0963	-0.0003	0.811	no clear effect
Fixed interval width	3.857 ± 0.110	+0.024	0.0964	-0.0003	0.820	no clear effect
Sigma_nwp only	3.786 ± 0.086	-0.047	0.0947	-0.0019	0.813	no clear effect
Sigma_expert only	3.832 ± 0.093	-0.001	0.0956	-0.0010	0.818	no clear effect
Single expert	3.874 ± 0.066	+0.040	0.0981	+0.0015	0.820	no clear effect
No regime weights	3.778 ± 0.097	-0.056	0.0954	-0.0013	0.809	no clear effect

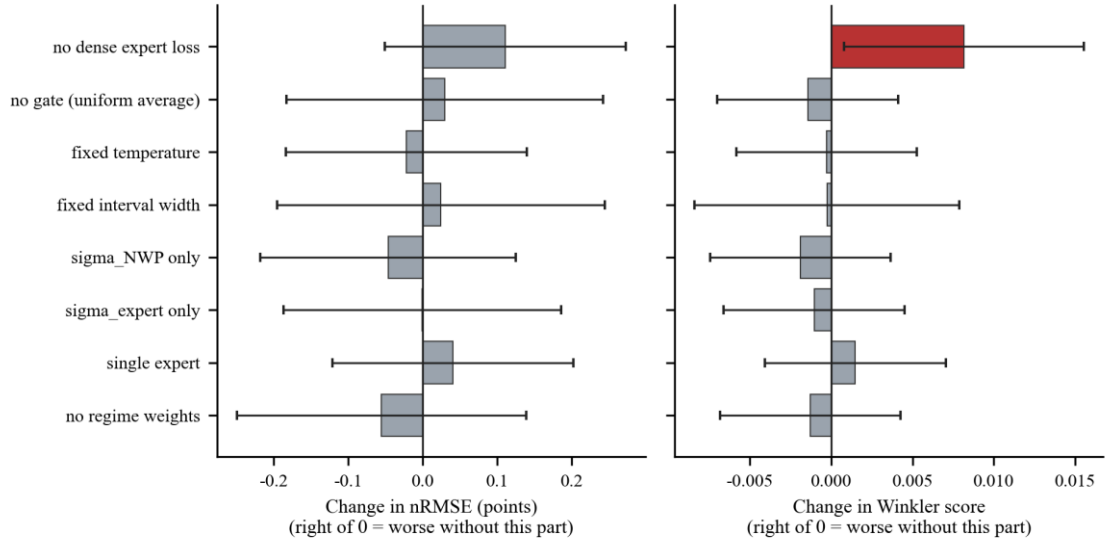


Figure 7.4: Change in nRMSE and Winkler score when a part is removed (grey = within noise; red = clearly worse; whiskers = ±2 standard deviations).

Only the dense expert loss showed a clear contribution: without it, nRMSE rose from 3.83% to 3.94%, the Winkler score rose by about 8%, and the coverage fell (all-points PICP from 0.820 to 0.776). All other parts (the gate, the temperature control, the σ-dependent interval width, using only one of the two uncertainty signals, using one expert instead of four, and the extra training weight on each expert's own regime) made no measurable difference on this data. We therefore **do not claim** that routing or regime specialisation improves accuracy. The gate is not uniform in the final model (Figure 7.8): its mean entropy is 0.86 against 1.39 for uniform routing, and the Dawn-ramp expert has the largest share. But it does not follow the weather type; for clear-midday cases the Clear-peak expert receives only 0.06 of the weight while the Dawn-ramp expert receives 0.49. The experts behave more like a small ensemble than like regime specialists.

We also tested other changes (Table 7.6). None of them improved on EMA alone: a larger weight decay, a smaller learning rate, an extra squared-error term on the median, and feeding the regime and volatility metadata into the shortcut all lay within the run-to-run variation of the reference, and combining them was no better than EMA alone.

Table 7.6: Other training options tested (Yorkshire, three seeds each).

Change	Runs	nRMSE %	Winkler
Reference (no EMA)	3	3.833 ± 0.081	0.0966
EMA weight averaging (decay 0.999)	3	3.697 ± 0.016	0.0922
Weight decay 1e-2	3	3.783 ± 0.110	0.0957
Learning rate 5e-4	3	3.883 ± 0.211	0.0976
Extra squared-error term on P50	3	3.850 ± 0.080	0.0983
Routing-metadata inputs	3	3.802 ± 0.168	0.0960
All four combined	3	3.730 ± 0.033	0.0946

7.5 Calibration and Coverage

About 59% of the forecast points lie in daylight. At night the true output and the interval are

both zero, so almost every night point counts as 'covered' (99.7%). The all-points PICP of the raw model was therefore 0.804, apparently on target, while the daytime coverage was only 0.670. Daytime coverage is the meaningful figure. Per-regime conformal calibration fitted on daytime validation points (Section 5.1.7) brought it to the within about one percentage point of the target on both measured regions (Table 7.7, Figures 7.5 and 7.6).

Table 7.7: Effect of conformal calibration on one trained model (seed 42) per site; target coverage 0.80.

Site	Daytime coverage raw	Daytime coverage calibrated	Winkler raw	Winkler calibrated	PINAW raw	PINAW calibrated
Yorkshire	0.670	0.794	0.0929	0.0891	0.0486	0.0594
South West	0.728	0.798	0.0638	0.0629	0.0365	0.0413
Vellore (simulated)	0.704	0.745	0.0873	0.0861	0.0438	0.0469

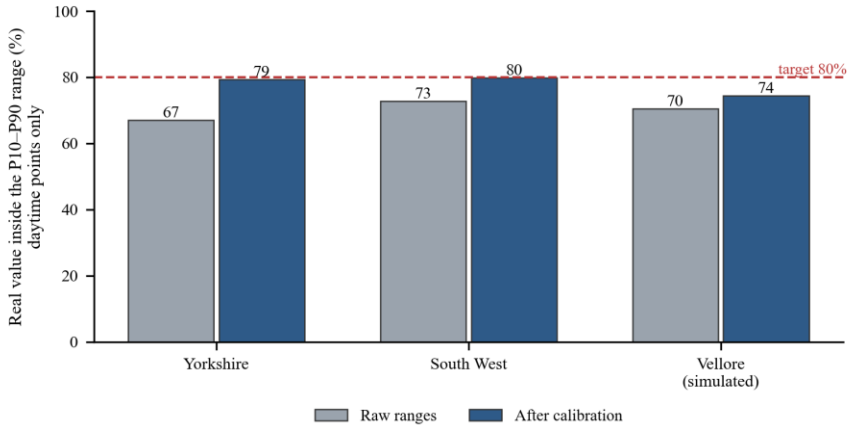


Figure 7.5: Share of daytime real values inside the P10–P90 range before and after calibration.

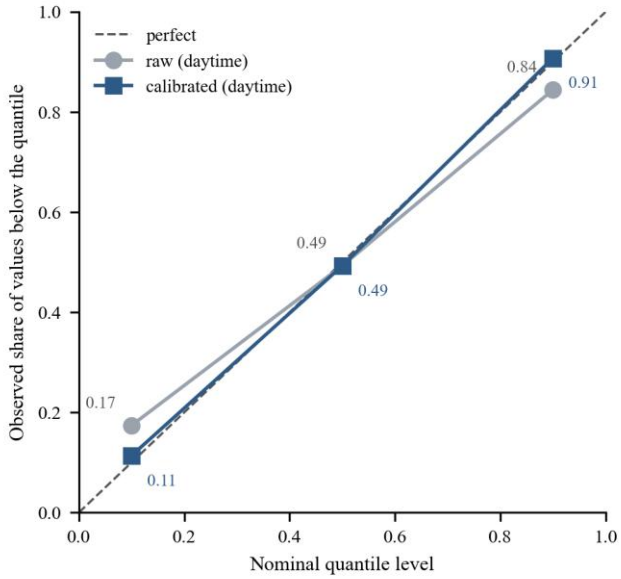


Figure 7.6: Reliability of the P10, P50 and P90 quantiles on daytime points of the Yorkshire test set.

Calibration widens the ranges (the Yorkshire PINAW rises from 0.0486 to 0.0594) but lowers the Winkler score slightly, because the reduction in missed observations outweighs the extra width. Two cautions apply. The volatile regime has only 44 validation windows, so its adjustment is noisy; and the validation and test periods fall in different seasons, so the

coverage on new data may differ a little from the target. On the simulated Vellore site calibration only reached 0.74, so we describe those ranges as still somewhat too narrow.

Table 7.8 gives accuracy and daytime coverage by weather situation for the main Yorkshire model, with LightGBM (seed 42) for comparison.

Table 7.8: Results by weather situation (Yorkshire, seed 42, test set).

Weather situation	Windows	nRMSE % ATERNET	nRMSE % LightGBM	Daytime coverage raw	Daytime coverage calibrated
Morning ramp-up	352	4.13	4.14	0.66	0.79
Clear sunny midday	459	4.04	4.13	0.67	0.83
Cloudy / overcast (incl. night)	1446	3.36	3.36	0.67	0.78
Patchy, changing clouds	205	4.28	4.22	0.68	0.80

ATERNET and LightGBM are close in every situation; the largest errors occur in the volatile and ramp situations, as expected. The 'overcast' group includes night-time windows, which is why it is the largest.

7.6 Behaviour of the Forecasts and of the Uncertainty

Figure 7.7 shows one example for each weather situation. The best guess follows the real output and the shaded range widens where the output is harder to predict, as in the patchy-cloud case.

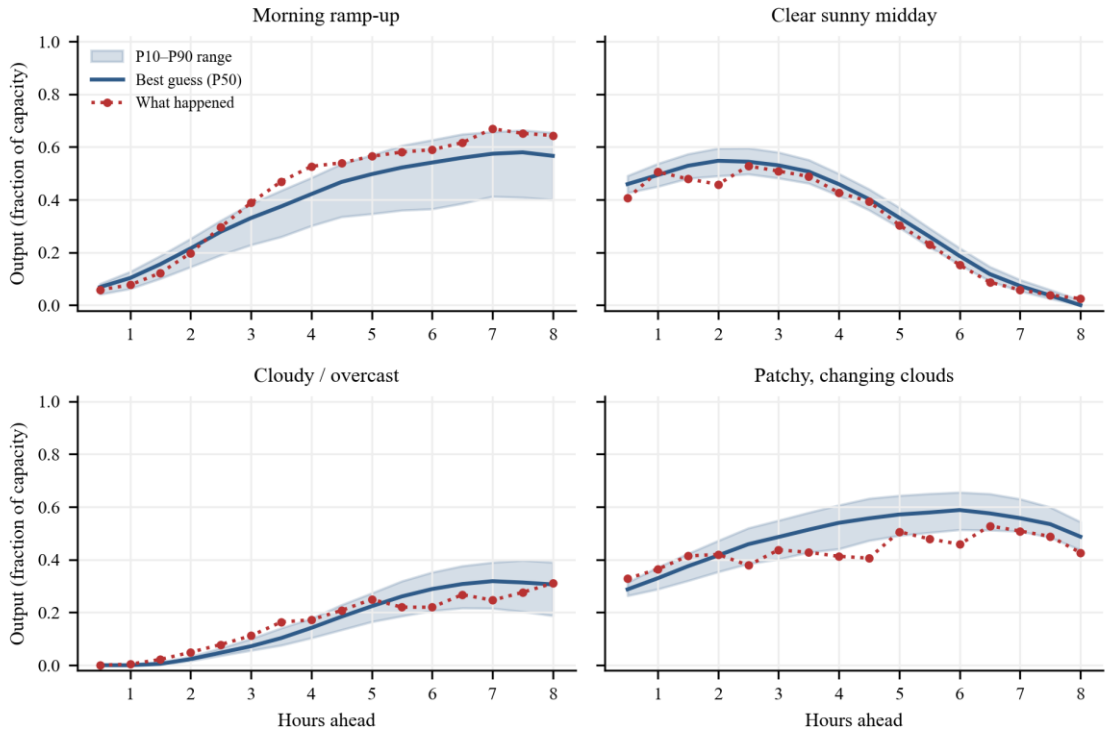


Figure 7.7: Example forecasts for each weather situation (Yorkshire test set).

Figure 7.8 shows how the gate distributes its trust among the experts in each real weather situation. A specialised pool would show a strong diagonal (each situation relying on its own expert); the figure shows that it does not, in agreement with the ablation results of Section 7.4.

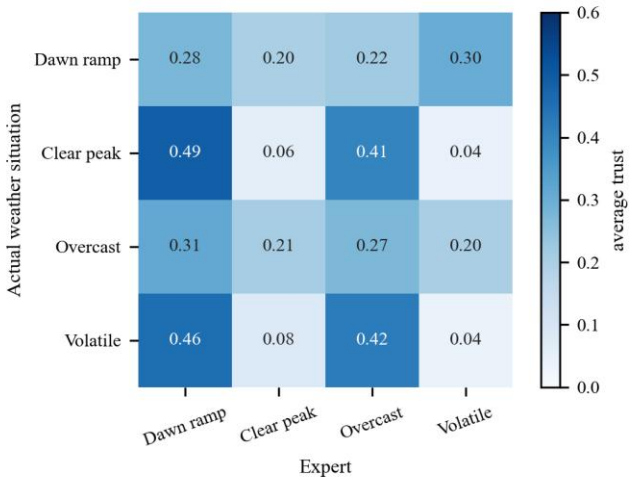


Figure 7.8: Average weight given to each expert (columns) in each real weather situation (rows) on the test set.

The model's unsureness is informative: the combined uncertainty σ_total has a correlation of 0.69 with the real error of the forecast, and the width of the forecast range correlates 0.82 with it (Figure 7.9). In other words, wider ranges are really drawn where the errors are larger.

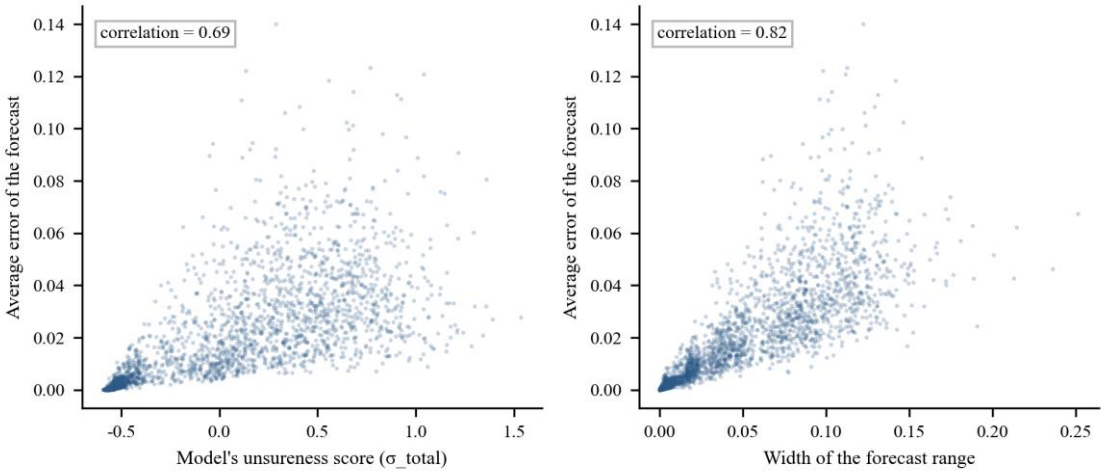


Figure 7.9: Average forecast error against the model's unsureness score and against the range width.

Figure 7.10 shows the error by forecast horizon. In the first hour ATERNET and LightGBM are equal (2.91% and 2.92% at one hour). ATERNET is more accurate from about one and a half to five hours ahead (for example 3.49% against 3.62% at two hours), the two are similar around five to six hours, and at the longest horizons ATERNET is slightly worse (4.06% against 3.92% at eight hours). The long-horizon error is dominated by the quality of the weather forecast.

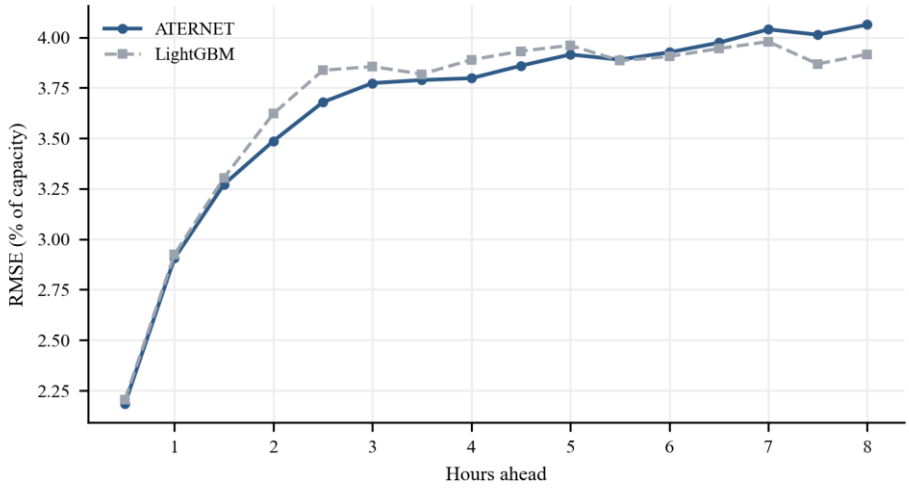


Figure 7.10: RMSE by forecast horizon for ATERNET and LightGBM (seed 42, Yorkshire).

Training (Figure 7.11) stopped automatically after 42 epochs with the best validation loss at epoch 33; an epoch took about 12 seconds on the CPU.

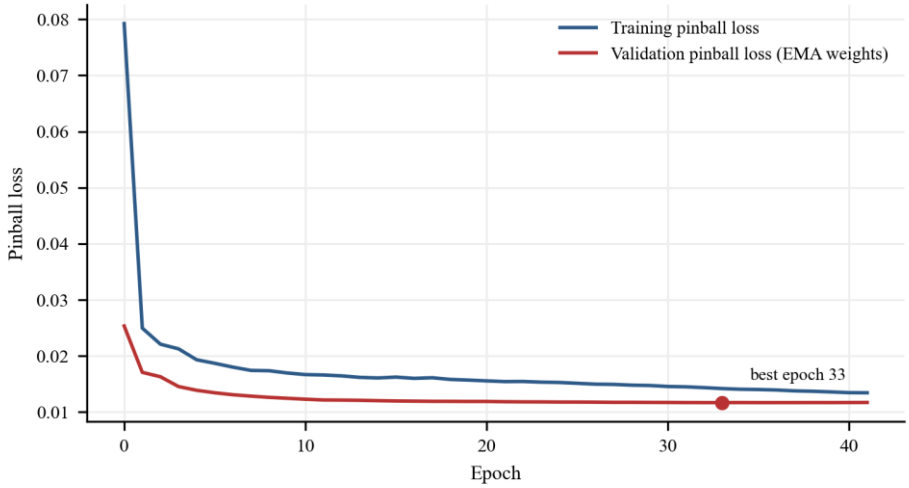


Figure 7.11: Training and validation loss of the main model.

7.7 Out-of-Sample Backtest

To test the complete live pipeline, forecasts were issued six times per day for 1–16 October 2026, after the end of the training period, using only data available at each issue time, and scored against what PV_Live later reported (Table 7.9, Figure 7.12).

Table 7.9: Backtest of the live pipeline, Yorkshire, 1–16 October 2026.

Quantity	Value
Forecasts issued / half-hours scored	96 / 1,536
nMAE / nRMSE (% of capacity)	2.62 / 4.08
R^2	0.940
Daylight MAE	42.2 MW (mean output 272 MW)
Day-ahead persistence daylight MAE	147.2 MW
Skill against persistence	0.704
Coverage of the P10–P90 range (all points)	0.749

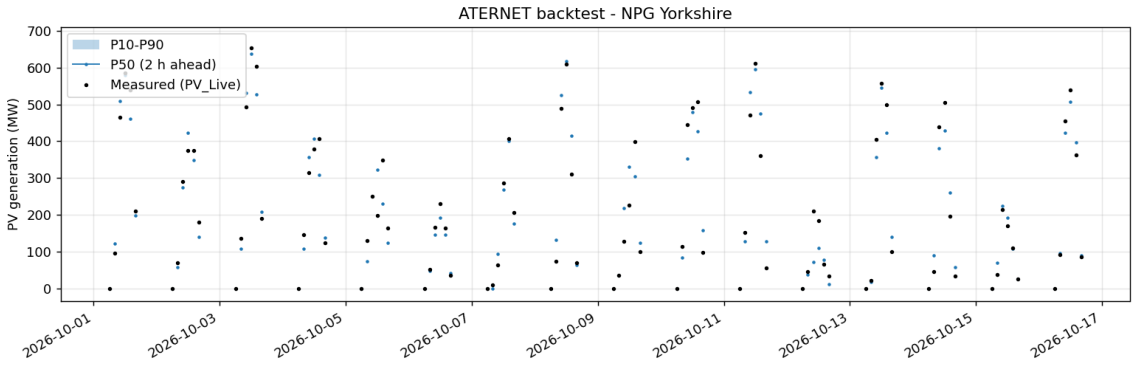


Figure 7.12: Backtest: P10–P90 range and best guess two hours ahead against the measured output.

The daylight mean absolute error is 42 MW, about 3.5 times better than persistence (147 MW). The range covered 75% of the points, somewhat below 0.80, which is consistent with the slightly narrow raw ranges found in Section 7.5. Two caveats apply: the historical weather archive is slightly better than a single live forecast, so live errors may be a little larger, and PV_Live is a regional aggregate that is smoother than one plant.

7.8 India Demonstration with Simulated Data

No open measured PV record exists for the VIT Vellore area, and we could not obtain data from the campus, so a demonstration site was built (Section 5.1.11). **All numbers in this section describe how well the model learns a simulated plant from real weather forecasts. They are not measured results and must not be read as real-world accuracy for India.** Training used January 2024 to September 2026 and four weather models (the fifth does not cover Vellore before 2025).

Table 7.10: Simulated Vellore plant (1 MWp), one training run each; test windows: 1,805.

Model	nRMSE %	nMAE %	R^2	Winkler	PINAW
ATERNET (seed 42)	3.75	1.70	0.9736	0.0873	0.0438
LightGBM (seed 42)	3.96	1.88	0.9705	0.0960	0.0482

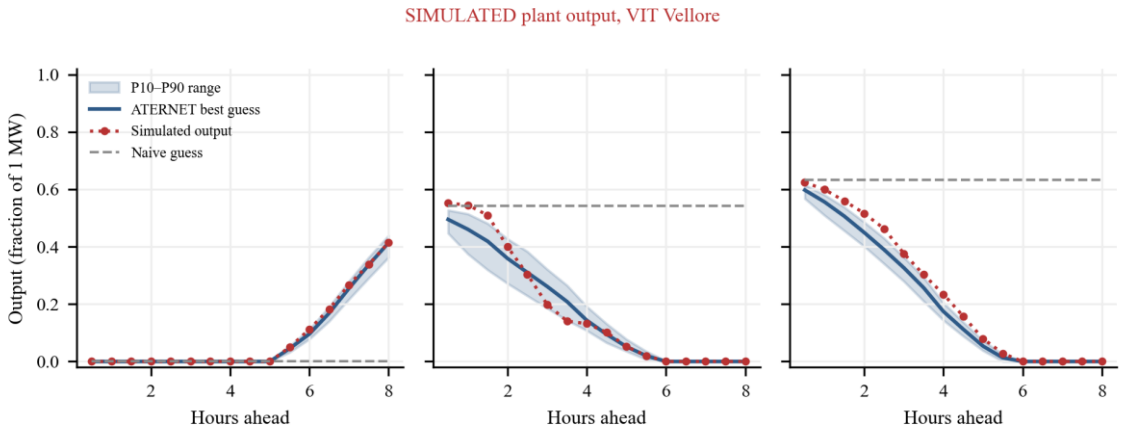


Figure 7.13: Simulated Vellore site: forecast ranges and the naive 'same as now' guess against the simulated output.

On this site ATERNET scored an nRMSE of 3.75% against 3.96% for LightGBM, and a lower Winkler score. Each model was trained only once, so this is the outcome of one run and is not a statistically tested win. Daytime coverage was 0.70 before and 0.74 after calibration,

i.e. short of the 0.80 target.

7.9 Additional Experiment: Denser Training Windows

Does more training data help? We built a training set with twice as many (overlapping) windows by halving the stride, keeping the validation and test sets identical so the results stay comparable, and re-ran ATERNET and LightGBM with seeds 42 and 43 (Table 7.11).

Table 7.11: Effect of doubling the number of training windows (Yorkshire).

Model	Stride 4 (seeds 42, 43)	Stride 2 (seeds 42, 43)
ATERNET nRMSE %	3.69, 3.68	3.75, 3.72
LightGBM nRMSE %	3.71, 3.72	3.68, 3.68

LightGBM improved with the extra windows, while ATERNET did not (it became slightly worse). With more windows LightGBM therefore pulls ahead on accuracy. The training schedule was not re-tuned for the larger set, so this does not prove that ATERNET cannot use more data, but it shows that the accuracy tie with LightGBM holds at the original data density and should not be generalised.

7.10 Discussion

- **What the evidence supports.** ATERNET is as accurate as the strongest baseline on Yorkshire, more accurate on South West England, stable across training runs, and its forecast ranges have a better Winkler score than every stable baseline. After calibration its daytime coverage is within about one percentage point of the 0.80 target on both measured regions.
- **What it does not support.** It does not show that adaptive routing or regime-specialised experts improve accuracy. Mixture-of-experts modules are used successfully in the related studies on PV, curtailment and thermal energy [15, 16, 17], but in our regional PV setting the ablations show they do not help; only the dense expert loss was shown to matter. It does not show that ATERNET beats a well-trained LSTM, or that it would stay ahead of LightGBM with more training data.
- **Threats to validity.** (i) Only two regions in England. (ii) Baselines were run with defaults and not tuned; tuning could narrow the gaps. (iii) Three seeds give only rough estimates of variation. (iv) Several design choices (such as EMA) were selected using results on the Yorkshire test period, which creates a mild selection bias there; the second region is the unbiased check. (v) The regime labels are rule-based and night-time windows default to 'overcast'. (vi) The India numbers are simulated.
- **Lessons.** Features that sound principled, such as an adaptive gate, must be tested against a plain average over several seeds; coverage must be measured where it matters (daylight); and single-run comparisons can be wrong by large amounts.

7.11 Cost Analysis

The project required no purchase of software, data or cloud services (Table 7.12). The costs are the time of the developer and the electricity used by a personal laptop; no monetary figure is claimed because none was recorded.

Table 7.12: Cost and resource summary.

Item	Cost / resource	Note
Software (Python, PyTorch, LightGBM, Streamlit, etc.)	Free, open source	Licences permit research use
PV data (Sheffield Solar PV_Live)	Free	Public API, no key
Weather data (Open-Meteo, ERA5)	Free	Public APIs; non-commercial use
Computing	One laptop (i5-12450H, 7.7 GB)	No GPU or cloud needed
Compute time	About 10 minutes per training run	More than 60 runs in total, of the order of 20 hours
Storage	A few GB	Datasets, checkpoints, cached downloads
Deployment (if adopted)	One small server or an existing PC	Model has about 968,000 parameters

For an operator, the benefit would come from reduced reserve requirements and better trading decisions; estimating it would need operator-specific data that we do not have, so this report makes no such claim.

CHAPTER 8
CONCLUSION

This project designed, built and evaluated ATERNET, a probabilistic solar power forecaster that combines a patch-based Transformer backbone, four regime experts, an uncertainty-aware router and a probabilistic decoder, together with a complete, tested pipeline from public data to an explanatory dashboard.

Main findings. (1) On measured regional data from Yorkshire, ATERNET reaches an nRMSE of 3.70 ± 0.02%, equal to LightGBM (3.72 ± 0.01%) and better than PatchTST, with a better Winkler score and very stable training. (2) On South West England, without re-tuning, it is ahead of both baselines (2.63 ± 0.01% against 2.69 ± 0.00% for LightGBM). (3) The original learned gate harmed accuracy and stability; a residual gate, a last-step linear shortcut and weight averaging were what produced the final accuracy. (4) Interval coverage measured over all hours was misleading; with daytime-only calibration the daytime coverage reaches 0.79 and 0.80 against a target of 0.80. (5) Of the architectural ideas, only the dense expert loss showed a measurable benefit; adaptive routing and regime specialisation did not improve accuracy.

Contributions. The project contributes a working probabilistic forecaster with built-in, tested guarantees; a reproducible multi-seed evaluation protocol with plain-language reporting; a worked example of diagnosing and correcting a harmful gate and a coverage statistic that overstated reliability; a clearly labelled simulated-site mechanism for places without data; and an understandable dashboard.

Limitations. The evidence comes from two English regions; baselines were not tuned; seed counts are small; the Indian demonstration is simulated; and LightGBM benefits more than ATERNET from additional training windows.

Future work. (i) Obtain measured PV data for an Indian site, for example from a campus plant, and retrain or fine-tune on it. (ii) Extend to more regions and seasons. (iii) Tune the baselines equally and test ensembles of several ATERNET runs. (iv) Make the experts specialise genuinely, for example with a diversity penalty or regime-specific data, and test again whether routing then helps. (v) Extend the horizon, add satellite-based inputs and deploy the forecaster as a service.

CHAPTER 9
REFERENCES

References are numbered in the order in which they are first cited (IEEE style).

[1] H.-Y. Lo, Y.-K. Wu, Q.-T. Phan, and W.-S. Tan, "A novel QR-based probabilistic forecasting method for solar power generation," IEEE Trans. Ind. Appl., vol. 61, no. 4, pp. 5381–5393, Jul./Aug. 2025, doi: 10.1109/TIA.2025.3546206.

[2] H. Zhao, Y. Zhang, W. Huo, F. Lin, and J. Wang, "A two-stage ensemble learning framework for short-term nonparametric probabilistic photovoltaic power forecasting integrating multi-source numerical weather predictions," IEEE Trans. Power Syst., vol. 41, no. 2, pp. 1231–1245, Mar. 2026, doi: 10.1109/TPWRS.2025.3613446.

[3] C. Lyu and S. Eftekharnejad, "Probabilistic solar generation forecasting for rapidly changing weather conditions," IEEE Access, vol. 12, pp. 79091–79103, 2024, doi: 10.1109/ACCESS.2024.3407778.

[4] R. Koenker and G. Bassett, "Regression quantiles," Econometrica, vol. 46, no. 1, pp. 33–50, 1978.

[5] T. Gneiting and A. E. Raftery, "Strictly proper scoring rules, prediction, and estimation," Journal of the American Statistical Association, vol. 102, no. 477, pp. 359–378, 2007.

[6] T. He, Y. Xu, and Y. Guo, "A probabilistic forecasting method for photovoltaic power under complex weather conditions based on multi-resolution deep learning," in Proc. 2026 5th Int. Conf. Smart Grid and Green Energy (ICSGGE), 2026, pp. 125–130, doi: 10.1109/ICSGGE69348.2026.11508327.

[7] C. Huang, Y. Wang, Z. Zhen, F. Xu, F. Wang, X. Liu, X. Xie, and Q. Fang, "Spatial-temporal attention mechanism and graph convolutional networks based distributed PV ultra-short-term probabilistic power forecasting," IEEE Trans. Ind. Appl., early access, 2026, doi: 10.1109/TIA.2026.3660789.

[8] Q.-T. Phan, Y.-K. Wu, and Q.-D. Phan, "Enhancing one-day-ahead probabilistic solar power forecast with a hybrid transformer-LUBE model and missing data imputation," IEEE Trans. Ind. Appl., vol. 60, no. 1, pp. 1396–1408, Jan./Feb. 2024, doi: 10.1109/TIA.2023.3325798.

[9] Y. He, X. Chen, Y. Zhang, and X. Yang, "Day-ahead PV power probability density forecasting with GAN-QRCNLSTM based on high-resolution data reconstruction," IEEE Trans. Ind. Informat., vol. 22, no. 2, pp. 1428–1439, Feb. 2026, doi: 10.1109/TII.2025.3627108.

[10] W. Wang, Y. Wang, F. Wang, F. Xu, G. Feng, and Z. Zhen, "Ultra-short-term power probabilistic forecasting method for distributed photovoltaic based on joint distribution modeling," IEEE Trans. Ind. Appl., vol. 62, no. 1, pp. 26–36, Jan./Feb. 2026, doi: 10.1109/TIA.2025.3589961.

[11] J. Ye, B. Zhao, D. Liu, Q. Wei, and Y. Wang, "TADNet: Temporal attention

decomposition networks for probabilistic energy forecasting," IEEE Trans. Power Syst., vol. 39, no. 6, pp. 7190–7202, Nov. 2024, doi: 10.1109/TPWRS.2024.3380388.

[12] P. Zhao, W. Hu, D. Cao, Z. Zhang, Y. Huang, L. Dai, and Z. Chen, "Probabilistic multienergy load forecasting based on hybrid attention-enabled transformer network and Gaussian process-aided residual learning," IEEE Trans. Ind. Informat., vol. 20, no. 6, pp. 8379–8393, Jun. 2024, doi: 10.1109/TII.2024.3366946.

[13] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, "Adaptive mixtures of local experts," Neural Computation, vol. 3, no. 1, pp. 79–87, 1991.

[14] N. Shazeer et al., "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in Proc. International Conference on Learning Representations (ICLR), 2017.

[15] H. Wang, R. Duan, and X. Yang, "A robust ensemble framework with context-gated dynamic fusion and temporal residual correction for photovoltaic power forecasting," in Proc. 2025 9th Int. Conf. Electrical, Mechanical and Computer Engineering (ICEMCE), 2025, pp. 1022–1026, doi: 10.1109/ICEMCE68156.2025.11466687.

[16] M. Z. Heidrich, M. A. F. Reis, C. K. Yamaguchi, L. O. Seman, and S. F. Stefenon, "Patch-based transformer with mixture of experts and conformal inference for curtailment time series forecasting," IEEE Access, vol. 14, pp. 64144–64160, 2026, doi: 10.1109/ACCESS.2026.3686958.

[17] L. O. Seman, S. F. Stefenon, J. P. Matos-Carvalho, A. Nied, and K.-C. Yow, "End-to-end decomposition-aware neural architecture search with sparse expert transformers for thermal energy forecasting," IEEE Access, vol. 14, pp. 67765–67787, 2026, doi: 10.1109/ACCESS.2026.3689436.

[18] Y. Romano, E. Patterson, and E. J. Candès, "Conformalized quantile regression," in Advances in Neural Information Processing Systems 32, 2019, pp. 3543–3553.

[19] V. Jensen, F. M. Bianchi, and S. N. Anfinsen, "Ensemble conformalized quantile regression for probabilistic time series forecasting," IEEE Trans. Neural Netw. Learn. Syst., vol. 35, no. 7, pp. 9014–9025, Jul. 2024, doi: 10.1109/TNNLS.2022.3217694.

[20] Y. Wang, Z. Zhen, F. Wang, S. Sun, Y. Cheng, and J. Yu, "A distributed PV ultra-short-term power deterministic and probabilistic forecasting based on dynamic graph network with shape-amplitude criteria," IEEE Trans. Ind. Appl., vol. 62, no. 2, pp. 1863–1873, Mar./Apr. 2026, doi: 10.1109/TIA.2025.3604364.

[21] F. Lin, Y. Zhang, H. Zhao, W. Huo, and J. Wang, "Short-term probabilistic forecasting for regional PV power based on convolutional graph neural network and parameter transferring," IEEE Trans. Power Syst., vol. 40, no. 3, pp. 2724–2736, May 2025, doi: 10.1109/TPWRS.2024.3503288.

[22] K. Song, M. Kim, and H. Kim, "Graph-based large scale probabilistic PV power forecasting insensitive to space-time missing data," IEEE Trans. Sustain. Energy, vol. 16, no. 1, pp. 160–173, Jan. 2025, doi: 10.1109/TSTE.2024.3447023.

[23] R. Rastgoo, N. Amjady, R. Shah, and S. M. Muyeen, "A diffusion-based probabilistic

ultra-short-term solar power prediction using the sky image sequences," IEEE Access, vol. 13, pp. 149532–149553, 2025, doi: 10.1109/ACCESS.2025.3600713.

[24] K. Wang, T. Wang, S. Shan, W. Dou, J. Zhang, H. Wei, and K. Zhang, "Multimodal ensemble photovoltaic power forecasting incorporating diffusion-based satellite image prediction," IEEE Trans. Sustain. Energy, vol. 17, no. 2, pp. 1425–1436, Apr. 2026, doi: 10.1109/TSTE.2025.3619938.

[25] O. Abedinia, A. Ghasemi-Marzbali, M. Shafiei, B. Sobhani, G. B. Gharehpetian, and M. Bagheri, "Wind power forecasting enhancement utilizing adaptive quantile function and CNN-LSTM: A probabilistic approach," IEEE Trans. Ind. Appl., vol. 60, no. 3, pp. 4446–4457, May/Jun. 2024, doi: 10.1109/TIA.2024.3354218.

[26] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[27] A. Vaswani et al., "Attention is all you need," in Advances in Neural Information Processing Systems 30, 2017, pp. 5998–6008.

[28] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in Proc. International Conference on Learning Representations (ICLR), 2023.

[29] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in Advances in Neural Information Processing Systems 30, 2017, pp. 3146–3154.

[30] R. L. Winkler, "A decision-theoretic approach to interval estimation," Journal of the American Statistical Association, vol. 67, no. 337, pp. 187–191, 1972.

[31] T. Bachlechner, B. P. Majumder, H. H. Mao, G. W. Cottrell, and J. McAuley, "ReZero is all you need: Fast convergence at large depth," in Proc. Conference on Uncertainty in Artificial Intelligence (UAI), PMLR vol. 161, 2021, pp. 1352–1361.

[32] B. T. Polyak and A. B. Juditsky, "Acceleration of stochastic approximation by averaging," SIAM Journal on Control and Optimization, vol. 30, no. 4, pp. 838–855, 1992.

[33] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems 32, 2019, pp. 8024–8035.

[34] Streamlit Inc., "Streamlit documentation." [Online]. Available: https://streamlit.io (accessed Oct. 2026).

[35] Sheffield Solar, University of Sheffield, "PV_Live: near-real-time estimates of GB solar PV generation." [Online]. Available: https://www.solar.sheffield.ac.uk/pvlive/ (accessed Oct. 2026).

[36] Open-Meteo, "Open-Meteo weather forecast and historical forecast APIs." [Online]. Available: https://open-meteo.com (accessed Oct. 2026).

[37] H. Hersbach et al., "The ERA5 global reanalysis," Quarterly Journal of the Royal Meteorological Society, vol. 146, no. 730, pp. 1999–2049, 2020.

[38] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in Proc. International Conference on Learning Representations (ICLR), 2019.

APPENDIX A – Sample Code

Selected excerpts of the source code are given below to document the key mechanisms. The complete code is in the project repository.

A.1 Residual gate with uncertainty-dependent temperature (src/phase4/gating_network.py).

```
class GatingNetwork(nn.Module):
    def __init__(self, h_expert_dim: int = 128, h_sigma_dim: int = 32,
                 n_experts: int = 4, hidden_dim: int = 64, dropout: float = 0.1,
                 init_beta: float = 1.0, residual: bool = False):
        super().__init__()
        gate_input_dim = h_expert_dim + h_sigma_dim + 3  # + [sigma_nwp_norm, sigma_expert_norm, alpha]


        self.logit_net = nn.Sequential(
            nn.Linear(gate_input_dim, hidden_dim), nn.GELU(), nn.Dropout(dropout),
            nn.Linear(hidden_dim, n_experts),
        )
        self.raw_beta = nn.Parameter(torch.tensor(float(init_beta)))
        self.n_experts = n_experts


        # Residual (ReZero-style) gate: weights = softmax(delta * logits / T)
        # with delta initialised to 0, so training starts at exactly uniform
        # routing (1/K, i.e. the `no_gate` solution) and only departs from it
        # as far as it pays off in validation loss. Off by default so existing
        # checkpoints and behaviour are unchanged.
        self.residual = residual
        if residual:
            self.delta = nn.Parameter(torch.zeros(()))


    def forward(self, h_expert: torch.Tensor, h_sigma: torch.Tensor,
                fusion_out: dict) -> dict:
        """
        h_expert: (B, h_expert_dim), h_sigma: (B, h_sigma_dim)
        fusion_out: dict from UncertaintyFusion.forward()


        Returns dict with gate_weights (B, n_experts), temperature (B,),
        entropy (B,) -- entropy is a useful diagnostic exposed for
        evaluation/interpretability plots.
        """
        sigma_nwp_norm = fusion_out["sigma_nwp_norm"]
        sigma_expert_norm = fusion_out["sigma_expert_norm"]
        alpha = fusion_out["alpha"]
        sigma_total = fusion_out["sigma_total"]


        gate_input = torch.cat([
            h_expert, h_sigma,
            sigma_nwp_norm.unsqueeze(1), sigma_expert_norm.unsqueeze(1), alpha.unsqueeze(1),
        ], dim=1)


        logits = self.logit_net(gate_input)  # (B, n_experts)


        beta = nn.functional.softplus(self.raw_beta)
        log_t = torch.clamp(beta * sigma_total, -MAX_LOG_TEMPERATURE, MAX_LOG_TEMPERATURE)
        temperature = torch.exp(log_t)  # (B,), strictly positive and finite


        scaled = logits / temperature.unsqueeze(1)
        if self.residual:
            scaled = self.delta * scaled
        gate_weights = torch.softmax(scaled, dim=1)


        with torch.no_grad():
            p = gate_weights.clamp_min(1e-8)
            entropy = -(p * p.log()).sum(dim=1)


        return {
            "gate_weights": gate_weights,
            "temperature": temperature,
            "entropy": entropy,
            "logits": logits,
        }
```

A.2 Weight averaging (EMA) in the trainer (src/training/trainer.py).

```
    def _ema_update(self):
        # Warm-up: early steps use a smaller decay so the average isn't
```

```
        # dominated by the random initialisation.
        self.ema_steps += 1
        d = min(self.cfg.ema_decay, (1 + self.ema_steps) / (10 + self.ema_steps))
        with torch.no_grad():
            for k, v in self.model.state_dict().items():
                if v.dtype.is_floating_point:
                    self.ema_state[k].mul_(d).add_(v.detach(), alpha=1.0 - d)
                else:
                    self.ema_state[k].copy_(v)


    def _ema_weights(self):
        """Temporarily swap the EMA weights into the model (no-op if EMA is off)."""
        if self.ema_state is None:
            yield
            return
        backup = {k: v.detach().clone() for k, v in self.model.state_dict().items()}
        self.model.load_state_dict(self.ema_state)
        try:
            yield
        finally:
            self.model.load_state_dict(backup)
```

A.3 Daytime mask and conformity scores for calibration (src/evaluation/conformal.py).

```
def _conformity_scores(y: np.ndarray, p10: np.ndarray, p90: np.ndarray,
                       score: str = "pointwise") -> np.ndarray:
    """
    CQR conformity score, E = max( p10 - y, y - p90 ).


    "pointwise" (default): one score per (window, horizon step), so the
        calibrated quantile targets the same per-point coverage (PICP) that
        is reported.
    "window_mean": the original per-window average over steps. It averages
        signed violations, and since most steps sit inside the interval
        (negative values) the resulting adjustment is ~0 or even negative,
        which left clear_peak under-covered and shrank the volatile
        interval. Kept only to reproduce earlier results.
    """
    if score == "pointwise":
        return np.maximum(p10 - y, y - p90).ravel()
    if score == "window_mean":
        return np.maximum((p10 - y).mean(axis=1), (y - p90).mean(axis=1))
    raise ValueError(f"Unknown score '{score}'.")


def daytime_mask(ds: ATERNETDataset, threshold: float = 0.02) -> np.ndarray:
    """
    (N, horizon) bool: forecast-horizon steps where the (scaled) forecast GHI
    is above `threshold`. Built from a model INPUT, not the outcome, so
    selecting these points doesn't bias coverage. At night the true output
    and the interval are both ~0, which is "covered" almost by definition and
    inflates PICP (a ~40 % share of points), so calibration and the daytime
    coverage report use these points only.
    """
    T = ds.x_seq.shape[1]
    return (ds.x_nwp[:, T:, 0].numpy() > threshold)
```

A.4 Forward pass of ATERNET with the last-step shortcut (src/models/aternet.py).

```
    def forward(self, x_seq: torch.Tensor, x_nwp: torch.Tensor, x_sigma: torch.Tensor,
                return_attention: bool = False, meta: torch.Tensor = None) -> dict:
        emb = self.backbone(x_seq, x_nwp, x_sigma, return_attention=return_attention)
        pool_out = self.expert_pool(emb["h_expert"])
        router_out = self.router(
            emb["h_expert"], emb["h_sigma"], x_sigma,
            pool_out["sigma_expert"], pool_out["stacked"],
        )
        y_hat_routed = router_out["y_hat_routed"]
        if self.last_step_skip is not None:
            window = x_seq.shape[1]
            parts = [x_seq[:, -1, :], x_nwp[:, window:, :].flatten(1)]
            if self.n_meta_features:
                if meta is None:
                    raise ValueError("This model was built with n_meta_features>0; "
                                     "pass meta=<(B, n_meta_features) tensor>.")
                parts.append(meta)
            skip_in = torch.cat(parts, dim=1)
            y_hat_routed = y_hat_routed + self.last_step_skip(skip_in)


        quantiles = self.decoder(
            y_hat_routed, emb["h_expert"], emb["h_nwp"], router_out["sigma_total"],
        )


        out = {
            **quantiles,
```

```
            "y_hat_routed": y_hat_routed,
            "gate_weights": router_out["gate_weights"],
            "temperature": router_out["temperature"],
            "entropy": router_out["entropy"],
            "alpha": router_out["alpha"],
            "sigma_nwp": router_out["sigma_nwp"],
            "sigma_expert_calibrated": router_out["sigma_expert_calibrated"],
            "sigma_total": router_out["sigma_total"],
            "expert_predictions": pool_out["predictions"],
            "expert_stacked": pool_out["stacked"],
            # h_expert is needed by the trainer to compute the dense
            # per-expert auxiliary loss without a second backbone pass.
            "h_expert": emb["h_expert"],
            "pool_out": pool_out,
        }
        if return_attention:
            out["nwp_attention_weights"] = emb["nwp_attention_weights"]
        return out
```

A.5 Simulated PV model for sites without measurements (src/phase1/simulated_pv.py).

```
def pv_power_kw(ghi: np.ndarray, temp_c: np.ndarray, peak_kw: float,
                soiling: Optional[np.ndarray] = None) -> np.ndarray:
    """Physical PV model (see module docstring). Inputs in W/m^2 and deg C."""
    ghi = np.clip(np.asarray(ghi, dtype=np.float64), 0.0, None)
    t_cell = np.asarray(temp_c, dtype=np.float64) + NOCT_COEFF * ghi
    derate = 1.0 + TEMP_COEFF * (t_cell - 25.0)
    soil = 1.0 if soiling is None else soiling
    p_dc = peak_kw * (ghi / 1000.0) * derate * SYSTEM_LOSSES * soil
    return np.clip(p_dc, 0.0, peak_kw / DC_AC_RATIO)
```